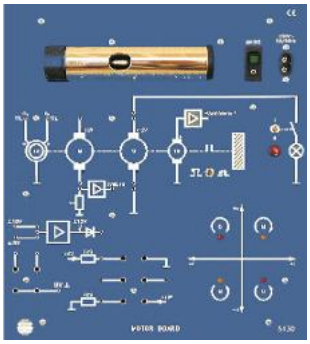
Laborversuch Regelungstechnik / Systemtechnik 2: Motorboard – Teil 1

Hochschule Esslingen
Fakultät Informationstechnik
Labor Regelungstechnik 2
Versuch: Motorboard

Bitte verwenden Sie
Matlab 2026a



Der Laborbericht ist spätestens eine Woche nach dem Labortermin abzugeben. Mangelhafte Vorbereitung beim Labortermin, verspätete oder unvollständige Abgabe des Laborberichts führen zum Nichtbestehen des Labors. Das Labor muss dann im folgenden Semester wiederholt werden.

Mit der Abgabe dieses Laborberichts und Ihrer Unterschrift bestätigen Sie, dass Sie den Bericht und die zugrundeliegenden Berechnungen, Programme und Simulationen selbständig ohne Hilfe Dritter angefertigt haben.

Laborbericht von: _____

Laborpartner: _____

Motorboard Nr.: _____ Labortermin: _____

Die Laborteilnehmer sollten jeweils ihre **eigene handschriftlich** ausgearbeitete Laborvorbereitung in die Laborveranstaltung mitbringen.

Beachten Sie im Labor die aushängenden Sicherheitsvorschriften und die Sicherheitsunterweisung.

© Professoren W. Lindermeir, T. Rothermel, W. Zimmermann

Inhaltsverzeichnis

1 Inhalt, Ziel und Ablauf des Versuchs 4

2 Versuchsaufbau 6

3 Beschreibung der verwendeten Komponenten 8

3.1 Motorboard HPS 5130 8

3.1.1 Gleichstrommotor M und Generator G 9

3.1.2 Strom-, Drehzahl- und Positionsmessung 9

3.1.3 Vierquadrant–Anzeige 11

3.2 Mikrocontrollerboard und Signalanpassschaltungen 13

3.2.1 Struktur des Hauptprogramms 13

3.2.2 PWM–Modul und Motoransteuerung 14

3.2.3 Repeating Timer und ADC–Messung von Strom– und Tachospannung . 15

4 Modellierung und Parameteridentifikation 17

4.1 Simulationsmodell für die Strecke 17

4.2 Parameteridentifikation 25

4.2.1 Elektrischer Teil des Motors: R, L 25

4.2.2 Reibung 26

4.2.3 Trägheitsmoment 26

4.2.4 Motorkonstante k_M 28

4.3 Validierung des Simulationsmodells, Drehzahlmessgenauigkeit 29

5 Auslegung des Drehzahlreglers in der Simulation 31

5.1 Dimensionierung des PI–Drehzahlreglers 31

5.2 Verifikation des Drehzahlreglers in der Simulation 32

6 Implementierung des PI–Reglers auf dem Mikrocontroller 32

6.1 Umwandlung in einen zeitdiskreten PI–Algorithmus 33

6.2 Test des zeitdiskreten PI–Algorithmus in C im Simulink–Modell 33

6.3 Portieren des zeitdiskreten PI–Algorithmus auf das Mikrocontroller–Board 34

© Professoren W. Lindermeir, T. Rothermel, W. Zimmermann

7 Positionsregler **34**

7.1 Dimensionierung und Test des Positionsreglers mit Matlab/Simulink 34

7.2 Implementierung der Positionsregelung auf dem Mikrocontroller–Board 35

8 Anhang: Analyse von Messdaten **37**

8.1 Software-Oszilloskop 37

8.2 Bedienung des Digital–Speicher–Oszilloskops 38

9 Anhang: Bedienoberfläche des Mikrocontroller–Programms **39**

Hinweis:
Die Simulink–Modelle für diesen Laborversuch und die hier abgebildeten Diagramme wurden mit Matlab R2026a erstellt. Falls Sie eine neuere Version von Matlab verwenden, speichern Sie die Simulink–Modelle bitte in der Version R2026a und bringen Sie diese Dateien dann ins Labor mit.

© Professoren W. Lindermeir, T. Rothermel, W. Zimmermann

1 Inhalt, Ziel und Ablauf des Versuchs

Der Laborversuch Motorboard findet aufgeteilt in zwei Laborterminen statt.

Thema: Entwurf einer Drehzahl- sowie einer Lageregelung in Kaskadenstruktur für einen Gleichstrommotor. Der Regler wird dabei mit Hilfe eines RP2040–Mikrocontrollers realisiert.

Inhalt:
- Analyse der vorgegebenen Systemkomponenten
- Modellbildung und Parameteridentifikation
- Validierung des Simulationsmodells
- Reglerentwurf und Inbetriebnahme auf dem Mikrocontroller
- Realisierung der Regler mit automatischer Codegenerierung und in Integerarithmetik

Ziel: In diesem Laborversuch werden die folgenden Inhalte der Vorlesung Regelungstechnik vertieft und praktisch angewendet:
- Modellbildung
- Simulation, Parameteridentifikation und Validierung des Simulationsmodells
- Reglerentwurf und -realisierung

Ablauf des Laborversuchs

Die Laborversuche bestehen jeweils aus einem Vorbereitungsteil und einem Teil, der erst im Labor durchgeführt wird. Die Vorbereitungsteile enthalten eine umfangreiche Anzahl von Aufgaben, die vor dem Laborversuch bearbeitet werden müssen und in den Umdrucken mit **VORBEREITUNG** gekennzeichnet sind. Bei der Vorbereitung sind sowohl Aufgaben und Berechnungen manuell zu lösen als auch Simulationen mit Matlab/Simulink durchzuführen.

1. **Notwendige Vorkenntnisse und Vorbereitungsaufwand**: Für die Vorbereitung dieses Laborversuchs muss ausreichend Zeit eingeplant werden. Insbesondere benötigen Sie für dieses Labor solide Kenntnisse im Umgang mit Matlab/Simulink aus vorhergehenden Laborversuchen der Vorlesungsreihe Regelungstechnik sowie Kenntnisse zum Mikrocontroller RP2350 aus der Vorlesung Computerarchitektur (der hier verwendete RP2040 ist dem RP2350 sehr ähnlich).

2. **Bearbeitung des Laborumdrucks**: Den Laborumdruck sowie vorbereitete Matlab/Simulink–Dateien finden Sie im Moodle–Kurs zur Vorlesung. Tragen Sie im Umdruck an den dafür vorgesehenen Stellen Ihre Lösungen handschriftlich in die freien Felder ein und ergänzen Sie bei Bedarf durch Zusatzblätter. Bringen Sie zum Laborversuch sämtliche Vorbereitungsunterlagen inklusive der Simulationsdateien mit. Berechnungen sollen handschriftlich, die Simulationsergebnisse müssen ausgedruckt und beschriftet vorliegen!

© Professoren W. Lindermeir, T. Rothermel, W. Zimmermann

Die Vorbereitung sollte in der Laborgruppe gemeinsam so durchgeführt werden, dass jeder Gruppenteilnehmer in der Lage ist, die Lösung zu sämtlichen Vorbereitungsaufgaben selbständig zu erklären und bei Bedarf sämtliche Simulationsprogramme vorzuführen und die Ergebnisse zu erläutern.

Mangelhafte Vorbereitung oder Nicht–Teilnahme führt zum Nicht–Bestehen des Labors. Das Labor kann in diesem Fall im folgenden Semester wiederholt werden.

3. **Projektverzeichnis im Labor**: Legen Sie bitte zu Beginn des Labors auf der **D:–Festplatte** des Laborrechners (nicht C:–Laufwerk!) ein temporäres Verzeichnis an, in das Sie alle Ihre Daten aus Ihrer Laborvorbereitung kopieren. Der Zugriff auf lokale Verzeichnisse ist üblicherweise deutlich schneller als auf Netzlaufwerke. Bitte vergessen Sie am Ende des Labors nicht, Ihre Datein zu sichern.

4. **Laborbericht**: Jede Studentengruppe gibt spätestens eine Woche nach dem Versuch einen vollständigen Bericht zum Labor ab. Dies kann bei sorgfältiger Vorbereitung schon direkt nach dem Labortermin erfolgen. Eventuell müssen jedoch vor der Abgabe einzelne Punkte noch nachgearbeitet werden.

   Der Laborbericht umfasst diesen Laborumdruck mit den handschriftlich (!) eingetragenen Lösungen sowie die Zusatzblätter mit den beschrifteten und kommentierten Simulationsergebnissen sowie gegebenenfalls Nebenrechnungen. Bei allen Unterlagen muss eindeutig erkennbar sein, zu welcher Aufgabe sie gehören, welche Signale (bei Simulationsergebnissen) dargestellt sind bzw. welche Parameter verwendet wurden. Diagrammachsen müssen beschriftet und skaliert werden. Bei Zahlenangaben sind die Einheiten mit anzugeben.

   Fassen Sie alle Unterlagen in einer pdf–Datei zusammen und laden Sie diese dann in den Moodle–Kurs hoch. Pro Laborgruppe ist genau ein Bericht abzugeben.

Beachten Sie bitte, dass die Vorbereitung und der Laborbericht nicht eine stupide Aneinanderreihung von Simulationsausdrucken sein sollten. Die Berechungen und vor allem die Simulationen sind nur dann sinnvoll, wenn Sie die Aufgabenstellung und den Lösungsweg verstanden haben und die Simulationsergebnisse interpretieren und kommentieren. Nur so können Sie das gewünschte Lernziel erreichen.

Sicherheitsvorschriften
Bitte beachten Sie die aushängenden Sicherheitsvorschriften im Labor und folgen Sie den Anweisungen der Laborbetreuer. Bei Fragen und Problemen wenden Sie sich an die zuständigen Laboringenieure, Herrn Zins oder Herrn Lehmann, bzw. an Prof. Rothermel oder Prof. Lindermeir.

© Professoren W. Lindermeir, T. Rothermel, W. Zimmermann

2 Versuchsaufbau

In diesem Kapitel soll ein Überblick über den Versuchsaufbau, die verwendeten Komponenten und ihre Verschaltung gegeben werden. Das Ersatzschaltbild der Strecke ist in Abbildung 1 dargestellt.

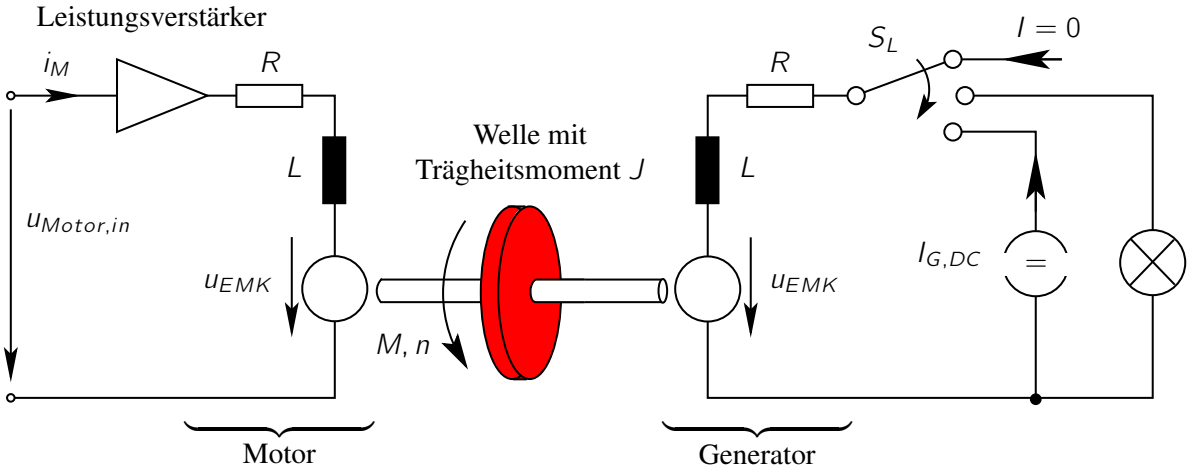


Abbildung 1: Ersatzschaltbild der Regelstrecke (Motorboard)

Im Bild links ist ein **Leistungsspannungsverstärker** und ein permanenterregter **Gleichstrommotor** abgebildet, der starr an eine Welle angekoppelt ist. Im rechten Teil ist ein Generator zu sehen, der baugleich mit dem Motor ist und ebenfalls starr an die Welle gekoppelt ist. Für den Motor gelten prinzipiell folgende Zusammenhänge:

- Das Drehmoment ist proportional zum Motorstrom, d.h. $M \sim i_M$
- Die induzierte Spannung ist proportional zur Drehzahl, d.h. $u_{EMK} \sim n$
  Über die angelegte Spannung $u_{Motor,in}$ kann man die Motordrehzahl steuern. Um die Drehrichtung umzukehren ($n < 0$), muss man eine negative Eingangsspannung $u_{Motor,in}$ anlegen.
- Wenn $u_{EMK} \sim n$ und $i_M \sim M$ dasselbe Vorzeichen haben, spricht man von **Motorbetrieb**, d.h. der Motor nimmt elektrische Leistung auf und gibt mechanische Leistung ab.
  Wenn $u_{EMK} \sim n$ und $i_M \sim M$ unterschiedliche Vorzeichen haben, spricht man von **Generatorbetrieb**, d.h. der Motor nimmt mechanische Leistung auf und gibt elektrische ab.
  Hinweis: Die beiden Spannungsquellen u_{EMK} sind in Verbraucherpfeilung dargestellt, d.h. Strom und Spannung sind in gleicher Richtung gepfeilt.

Der **Generator** erzeugt ein Belastungsmoment für den Antriebsmotor. Der Strom durch den Generator und damit das Belastungsmoment kann auf drei Arten beeinflußt werden:

- Falls der Schalter S_L oben ist, kann kein Generatorstrom fließen. Dadurch kann der Generator – abgesehen von Reibung – kein Drehmoment auf die Welle ausüben.

© Professoren W. Lindermeir, T. Rothermel, W. Zimmermann

- Falls der Schalter S_L in der mittleren Stellung ist, speist der Generator eine Glühlampe (= konstanter Widerstand). Dadurch erzeugt der Generator immer ein der momentanen Bewegungsrichtung entgegengesetztes Lastmoment an der Welle, d.h. er wirkt bremsend.
- Falls der Schalter S_L unten ist, wird durch eine geregelte Stromquelle ein vorgegebener Strom $I_{G,DC}$ in den Generator eingeprägt. Hierdurch kann ein definiertes Laststörmoment eingeprägt werden, das je nach Richtung des eingeprägten Stromes positiv oder negativ sein kann.

Zur Regelung verwenden wir einen RP2040 Mikrocontroller. Ein Übersichtsbild des gesamten Versuchsaufbaus ist in Abbildung 2 dargestellt.

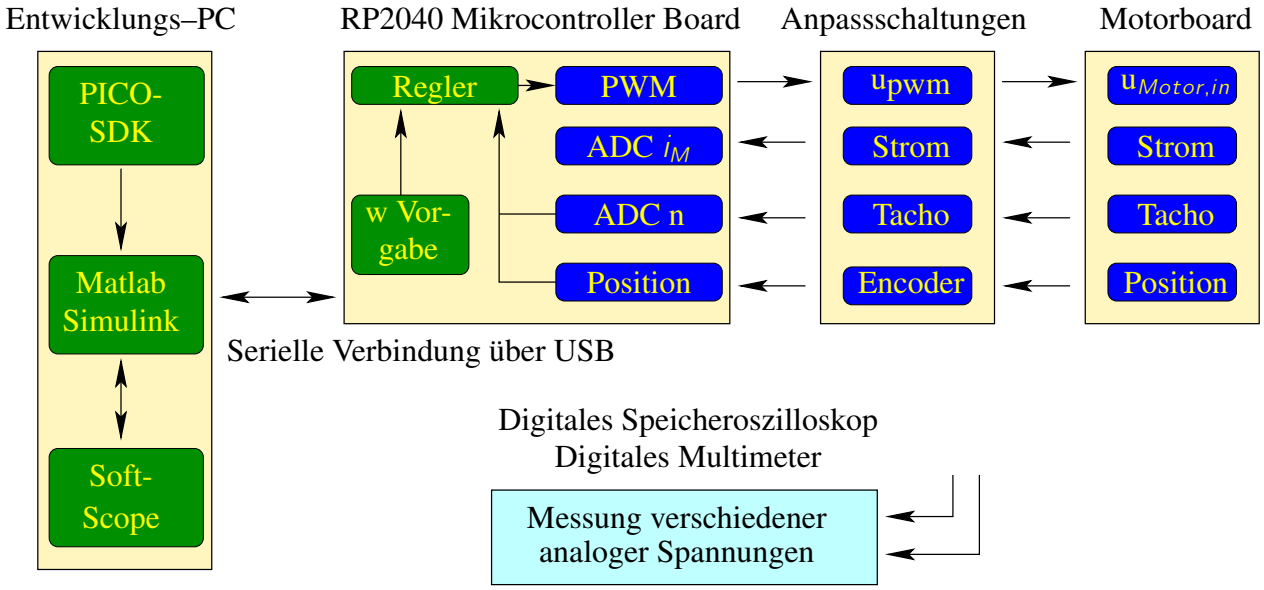


Abbildung 2: Blockschaltbild des gesamten Versuchsaufbaus.

Der Mikrocontroller steuert über seinen PWM–Ausgang die Eingangsspannung am Motorboard. Die Messgrößen sind der Strom i_M durch den Motor, die Drehzahl n und die Position φ der Welle. Diese Größen werden über Anpassschaltungen (Spannungsteiler, Verstärker) geführt. Im Mikrocontroller wird ein Regelalgorithmus in Kaskadenstruktur entworfen, der basierend auf den Messsignalen Drehzahl und Position die Ansteuerung der Motoreingangsspannung bestimmt. Der Motorstrom wird in dieser Anordnung nur gemessen, um eine Überlast des Motors zu erkennen und falls erforderlich abzuschalten.

Als zusätzliche Hardware–Komponenten werden ein digitales Speicheroszilloskop (DSO) sowie ein digitales Multimeter zur Messung von analogen Spannungen verwendet.

Der Mikrocontroller kann über das USB–Interface seine Standardausgabe an den Entwicklungs–PC senden. Über diese Verbindung werden zwei Signale in Echtzeit an Matlab weitergegeben und dort mit Hilfe eines Soft–Oszilloskops graphisch dargestellt. Dies ist die einzige Möglichkeit, die Position der Welle in der Entwicklungsumgebung am PC verfügbar zu machen. In Anhang 8.1 ist beschrieben, wie das entsprechende Vorgehen hierzu ist.

© Professoren W. Lindermeir, T. Rothermel, W. Zimmermann

Die Modellierung, Simulation, Reglerauslegung und der Vergleich zwischen Messung und Simulation findet am Entwicklungs–PC mit Matlab/Simulink statt.

Vorbereitungsaufgabe 1: Simulationsmodell

Zum besseren Verständnis der Anwendung können Sie das Simulink–Modell *Regelkreis_ZK.slx* öffnen.

- Bevor Sie die Simulation starten, führen Sie bitte zunächst das M–File *params_m.m* aus, um die Parameterwerte zu initialisieren, und stellen Sie die Schalter im Simulink–Modell so ein, dass das gewünschte Modell entsteht (Positionsregler, Drehzahlregler oder Handbetrieb). Beginnen Sie mit der Drehzahlreglung.
- Mit einem weiteren Schalter (im Modell des Motorboards) können Sie die Störgröße ein- oder abschalten. Die Lampe in Abbildung 1 ist hier nicht modelliert.
- Rechts oben befinden sich zwei Animationsblöcke. Diese können zur besseren Simulationsperformance ausgeschalten werden. Klicken Sie die Blöcke an und schalten Sie sie ein (Häkchen bei *Enable–CheckBox*) oder aus.
- Der Block Tacho stellt einen Drehzahlmesser dar, an dem Sie die aktuelle Drehzahl in U/min sehen können.
- Der Block Linearachse stellt eine Achse dar, die die rotatorische Bewegung der Welle in eine translatorische Bewegung umsetzt. An der Achse kann die Positionierung der Welle gut erkannt werden. Darüber hinaus kann über die Buttons eine Sollposition für den Positionsregler vorgegeben werden. Diese Vorgabe der Sollposition können Sie ausprobieren, wenn Sie (nur für diese Simulation) bei *Stop Time* inf eintragen.

Wenn Sie die Simulationen ausführen, sollten die Regler schon halbwegs sinnvolles Verhalten zeigen. Beide Regler schwingen relativ langsam ein. In diesem Laborversuch werden wir dies ändern. Zu dieser Vorbereitungsaufgabe müssen Sie nichts abgeben.

3 Beschreibung der verwendeten Komponenten

Da in diesem Laborversuch ein reales Regelungssystem betrachtet wird, werden zunächst die verwendeten Systemkomponenten untersucht. Im Anschluß daran findet die Modellbildung statt, die Voraussetzung für den Regelungsentwurf ist.

3.1 Motorboard HPS 5130

Das Motorboard HPS 5130 ist in Abbildung 3 dargestellt. Oben sehen Sie die Welle mit dem Motor und dem Generator. Die elektrischen Anschlüsse dieser Komponenten sind bei den jeweiligen Symbolen durch Stecker zugänglich. Den 3–fach Schalter aus Abbildung 1 realisieren wir wie folgt:

© Professoren W. Lindermeir, T. Rothermel, W. Zimmermann

Wicklungswiderstand	R	$11,9\Omega$
Induktivität	L	keine Angabe
Motorkonstante	k_m	$14,8 \cdot 10^{-3}\ \frac{Nm}{A}$
Nennspannung	U_N	$12V$
Leerlaufstrom bei $u = U_N$	I_0	$20mA$
Leerlaufdrehzahl bei $u = U_N$	n_0	$7800\frac{U}{min}$
Max. Dauerstrom	I_{max}	$580mA$

Tabelle 1: Parameter des Motors bzw. des Generators

- Schalter oben: Steckbrücke SB ziehen oder Schalter SL auf 0.
- Schalter mitte: Steckbrücke SB einstecken und Schalter SL auf 1.
- Schalter unten: Steckbrücke SB ziehen und externe Stromquelle beim unteren Kontakt einspeisen.

Das Trägheitsmoment der Welle kann über den Schalter SJ verändert werden. In Position 0 hat das Trägheitsmoment der Welle den Wert $J = J_{K(lein)}$. In Position 1 wird das Trägheitsmoment auf den Wert $J = J_{G(roß)}$ vergrößert (wird auf dem Motorboard elektronisch nachgebildet).

3.1.1 Gleichstrommotor M und Generator G

Der Motor und der Generator sind permanenterregte Gleichstrommotoren vom gleichen Typ. Die elektrischen Daten des Motors bzw. Generators aus dem Datenblatt des Herstellers finden Sie in Tabelle 1. Im Verlauf des Labors werden diese Größen teilweise nachgemessen.

3.1.2 Strom-, Drehzahl- und Positionsmessung

Stromsensor CS: Der Motorstrom i_M wird durch die Messung des Spannungsabfalls an einem Messwiderstand $R_{shunt} = 1\Omega$ (Englisch: Shunt) ermittelt und das Messignal mit einem Messverstärker mit $1V$ pro $100mA$ verstärkt.

Tachogenerator TG: Die Motordrehzahl n wird über einen Tachogenerator in eine proportionale Spannung gewandelt. Die Spannung am Verstärkerausgang beträgt $2V$ pro $1000\frac{U}{min}$.

Positionsgeber TX: Der Drehwinkel φ wird digital erfasst. Wenn sich die Welle dreht, erzeugt der Positionsgeber zwei Encoder–Rechtecksignale U_1 und U_2 (Abbildung 4), deren Phasen um 90° gegeneinander versetzt sind (Inkremental– oder Quadratur–Positionsgeber). Jedes Signal hat 100 Impulse je Umdrehung, insgesamt also 400 Signalflanken je Umdrehung. Die Messung ist inkrementell (relativ), eine absolute Nullposition existiert bei diesem Geber nicht.

Wir verwenden diesen Positionsgeber in zwei unterschiedlichen Anwendungen im Labor:

© Professoren W. Lindermeir, T. Rothermel, W. Zimmermann

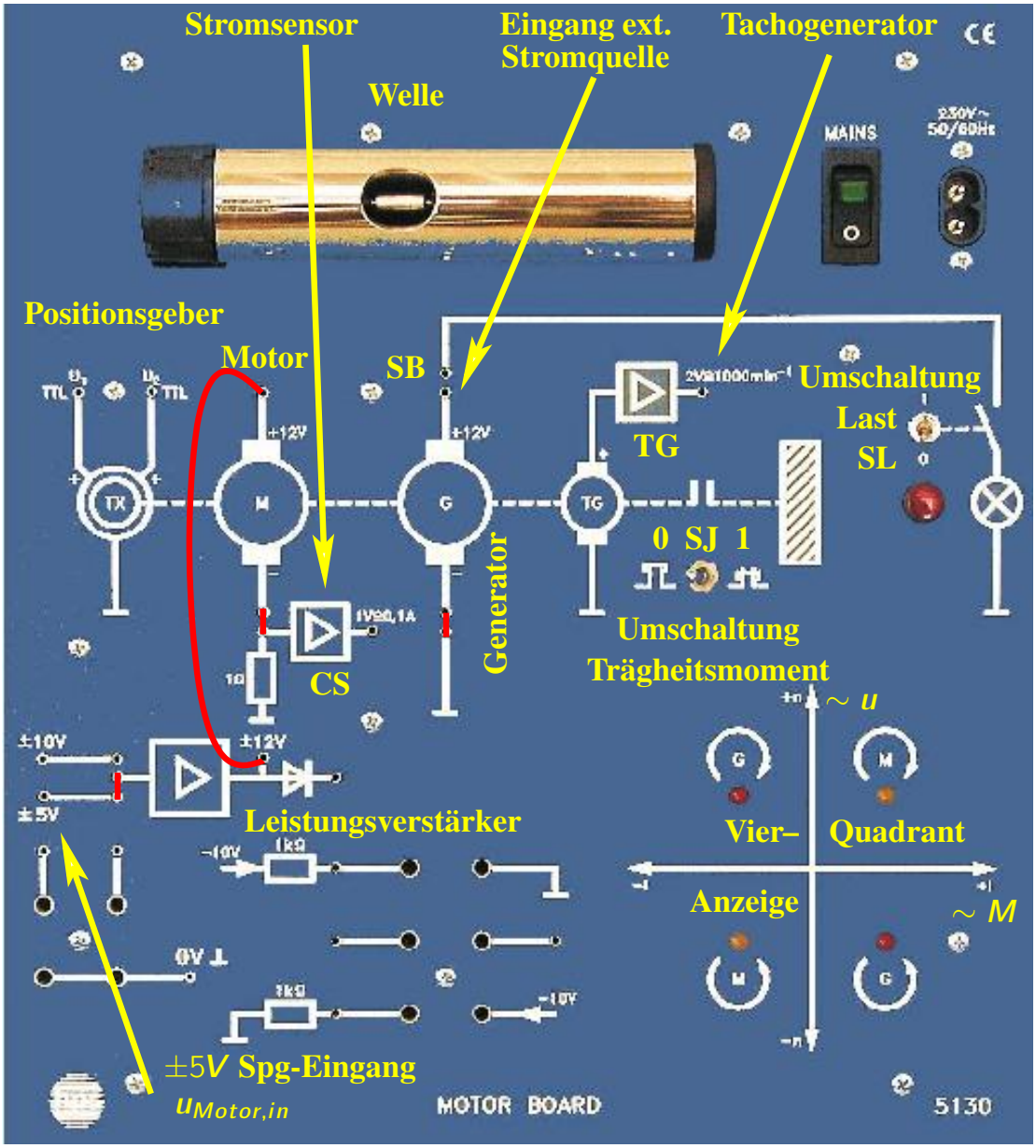


Abbildung 3: Motorboard 5130 der Firma HPS als Regelstrecke. Die offenen Buchsen sind im Labor bei Bedarf mit kleinen Steckbrücken oder Kabeln kurzgeschlossen (teilweise in rot angedeutet).

1. Falls U_1 bzw. U_2 mit einem Digitalspeicheroszilloskop gemessen und die Welle bei konstanter Drehzahl betrieben wird, kann aus der Periodendauer T_E der gemessenen Spannung auf die Drehzahl der Welle geschlossen werden (eine Umdrehung pro $100 \cdot T_E$).

2. Zur Positionsbestimmung der Welle ist im Mikrocontroller eine Komponente (*quadrature_encoder*) implementiert, die die positiven und negativen Flanken der beiden Signale U_1 und U_2 erkennt und eine Zählvariable jeweils inkrementiert (Vorwärtsfahrt) bzw. dekrementiert (Rückwärtsfahrt) und so den (relativen) Drehwinkel der Welle bestimmt. Die Drehrichtung wird dabei aus dem Pegel des jeweils anderen Signals ermittelt. Ist beispielsweise zum

© Professoren W. Lindermeir, T. Rothermel, W. Zimmermann

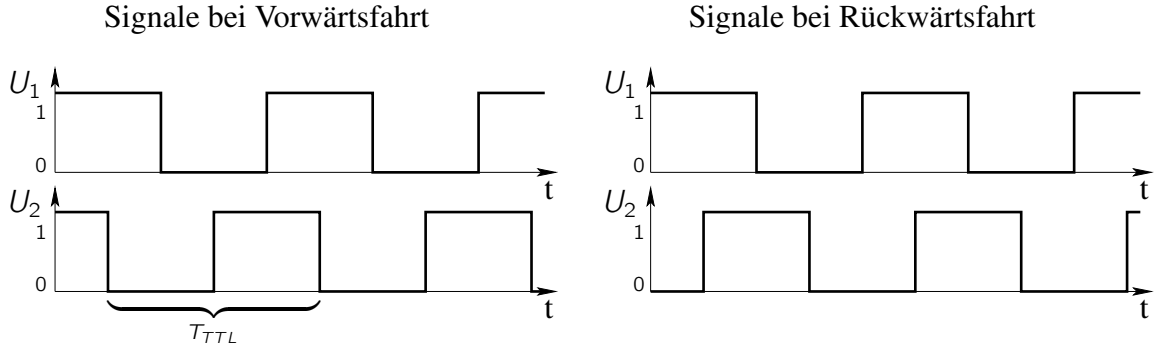


Abbildung 4: Quadratur–Ausgangssignale des Positionsgebers bei const. Drehzahl.

Zeitpunkt der steigenden Flanke des Signals U_1 der Pegel von U_2 high, so wird die Drehrichtung als positiv erkannt. Die Auflösung der Positionsmessung beträgt $360^0/400 = 0.9^0$. Das Problem, dass man lediglich relative Winkeländerungen erfassen kann, wird in Applikationen dadurch umgangen, dass man einen definierten Punkt anfährt (z.B. einen mechanischen Anschlag) und dort den inkrementellen Positionszähler initialisiert.

3. Der Quadratur–Encoder ist in einer Programmable I/O (PIO)–Statemachine des RP2040 implementiert und belastet die CPU nur minimal. Über das Zählen der Flanken und der dabei jeweils verstrichenen Zeit kann mit Hilfe dieser Komponente auch eine relativ genaue Drehzahl des Motors ermittelt werden, die genauer als die über den Tachogenerator ermittelte ist.

3.1.3 Vierquadrant–Anzeige

Unten rechts am HPS Motorboard befindet sich eine sogenannte Vierquadranten–Anzeige mit LEDs, die die Drehrichtung und die Richtung des Drehmoments anzeigt:

Drehmoment $M \sim i_M$	Drehzahl $n \sim u_{EMK}$	Betriebsart
> 0 (positiv)	> 0 (positive Drehrichtung)	Motorbetrieb (1.Quadrant)
< 0 (negativ)	> 0 (positive Drehrichtung)	Generatorbetrieb (2.Quadrant)
< 0 (negativ)	< 0 (negative Drehrichtung)	Motorbetrieb (3.Quadrant)
> 0 (positiv)	< 0 (negative Drehrichtung)	Generatorbetrieb (4.Quadrant)

Im Motorbetrieb wird elektrische Leistung $P_{elektr.} = u_{EMK} \cdot i_M$ in mechanische Leistung $P_{mech} = M \cdot 2\pi n$ umgesetzt, im Generatorbetrieb mechanische Leistung in elektrische Leistung (vergl. Abbildung 1). In beiden Fällen geht ein (hier relativ großer) Teil der Leistung als elektrische Verlustleistung $P_{V,el} = R \cdot i_M^2$ und als mechanische Verlustleistung durch (drehzahlabhängige) Reibung verloren.

© Professoren W. Lindermeir, T. Rothermel, W. Zimmermann

Vorbereitungsaufgabe 2: Berechnung der stationären Drehzahl des Motors

Berechnen Sie aus den Daten in Tabelle 1 und dem Ersatzschaltbild in Abbildung 1 die an den Motorklemmen aufgenommene elektrische Nennleistung $P_{el,N}$ bei Nennspannung und maximalem Dauerstrom. (Achtung: Bei allen Zahlenwerten Einheiten nicht vergessen!)

$P_{el,N} =$

Wie groß wird bei Nennleistung die elektrische Verlustleistung $P_{V,el,N}$ am Wicklungswiderstand?

$P_{V,el,N} =$

Im Leerlaufbetrieb müsste der Motorstrom eigentlich 0 sein. In Wirklichkeit fließen laut Datenblatt aber $I_0 = 20mA$. Rechnen Sie für den Leerlauf ebenfalls die an den Motorklemmen aufgenommene elektrische Leistung $P_{el,0}$ sowie die Verlustleistung $P_{V,el,0}$ am Wicklungswiderstand aus.

$P_{el,0} =$

$P_{V,el,0} =$

Die Differenz dieser beiden Leistungswerte entspricht ungefähr den mechanischen Reibungsverlusten $P_{mech,0}$. Man darf davon ausgehen, dass die mechanischen Verluste bei einem Elektromotor im Wesentlichen nur von der Drehzahl, aber nicht vom Drehmoment abhängen, d.h. das Reibungsmoment $M_R = d \cdot 2\pi n$ ist näherungsweise proportional zur Drehzahl (sogenannte viskose Reibung). Den Proportionalitätsfaktor, den sogenannten Reibbeiwert $d = \frac{P_{mech,0}}{(2\pi n_0)^2}$ kann man dann aus den Reibungsverlusten berechnen.

Die induzierte Spannung des Motors im Leerlauf bei Nennspannung läßt sich aus der Maschengleichung abschätzen: $U_{EMK,0} = U_N - R \cdot I_0 = 12V - 11,9\Omega \cdot 20mA = 11,76V$. Die Drehzahl n_0 im Leerlauf beträgt laut Datenblatt 7800U/min. Bei Nennspannung und max. Dauerstrom sinkt die induzierte Spannung auf $U_{EMK,N} = 12V - 11,9\Omega \cdot 580mA \approx 5,1V$ ab. Da die induzierte Spannung der Drehzahl proportional ist, kann man aus dem Spannungsverhältnis die Nenndrehzahl des Motors n_N ausrechnen, die im Datenblatt leider nicht angegeben wurde. Wie groß ist sie?

$n_N =$

Wegen des relativ großen Wicklungswiderstandes sinkt gerade bei kleinen Motoren die Drehzahl bei Belastung gegenüber dem Leerlauf sehr stark ab. Um dies zu verhindern, ist eine Drehzahlregelung notwendig, die dem Drehzahleinbruch durch eine passende Vergrößerung der Klemmenspannung entgegenwirkt.

© Professoren W. Lindermeir, T. Rothermel, W. Zimmermann

3.2 Mikrocontrollerboard und Signalanpassschaltungen

Ähnlich der Vorlesung *Computerarchitektur* verwenden wir hier den RP2040 Mikrocontroller, der dem RP2350 sehr ähnlich ist, mit der zugehörigen Entwicklungsumgebung. Nachfolgend sind die verwendeten Peripherie–Komponenten beschrieben, die wir als Interface–Schaltungen zum Motorboard benötigen.

Es werden drei verschiedene Mikrocomputer–Projekte verwendet:

- *Lab1_uC*: Regler in Gleitkomma–Arithmetik (wird in beiden Laborversuchen verwendet).
- *Lab2_uC.autocode*: Regler in Gleitkomma–Arithmetik und automatischer Code–Generierung (wird erst im zweiten Laborversuch benötigt).
- *Lab2_uC.integer*: Regler in Integer–Arithmetik (wird erst im zweiten Laborversuch benötigt).

3.2.1 Struktur des Hauptprogramms

Um den generellen Aufbau des vorgegebenen Hauptprogramms besser verstehen zu können, sollen Sie das Projekt *Lab1_uC* zunächst analysieren.

Vorbereitungsaufgabe 3: Analyse der Struktur des Hauptprogramms

In den Dateien *src/main.c* und *src/utils.c* finden Sie zwei Zustandsautomaten mit den Aufzählungstypen *control_state* und *display_state*, die den Betriebsmodus der Regelung (Drehzahlregelung, Lageregelung usw.) festlegen. Die Bedienung des Programms erfolgt über die Taster auf der Platine des Mikrocontrollerboards.

Ermitteln Sie zunächst für jeden der beiden Automaten die Namen der möglichen Zustände und in welcher Routine die Zustände verändert werden.

Geben Sie das sich ergebende Zustandsdiagramm für den *control_state*–Automaten in Abbildung 5 und für den *display_state*–Automaten in Abbildung 6 an. Sie werden bei der Analyse sehen, dass das Verhalten der Automaten u.a. von einem Fehlerzustand abhängt. Die Abhängigkeit vom Fehlerzustand können Sie in den Zustandsdiagrammen weglassen.

Wie kann ein Fehlerzustand quittiert werden:

Hinweis: Sehen Sie sich zur Bearbeitung dieser Vorbereitungsaufgabe auch die Beschreibung in Anhang 9 an.

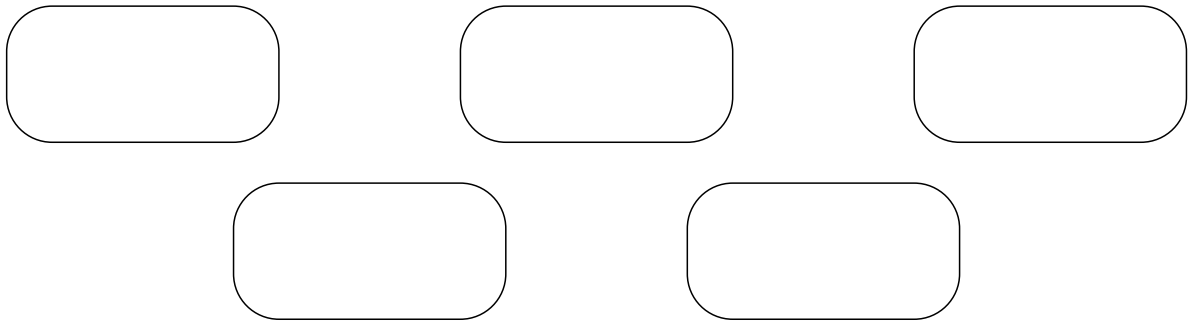
© Professoren W. Lindermeir, T. Rothermel, W. Zimmermann



Zustandsübergänge jeweils

Zustandsübergänge in implementiert

Abbildung 5: Zustandsdiagramm des *control_state*–Automaten



Zustandsübergänge jeweils

Zustandsübergänge in implementiert

Abbildung 6: Zustandsdiagramm des *display_state*–Automaten

3.2.2 PWM–Modul und Motoransteuerung

Um den Motor mit einer analogen Spannung $u_{Motor,in}$ ansteuern zu können, verwenden wir das PWM–Modul des Mikrocontrollers. Im Labor wird Pin *PICO_PWM_OUT* mit einer Periodendauer des PWM–Signals von $2.04\mu sec$ verwendet. In Abhängigkeit des 8–bit Wertes des Tastverhältnisses kann der Gleichanteil der Ausgangsspannung dabei von $0V$ (PWM–Wert = 0) bis $3.3V$ (PWM–Wert = 255) in 255 Schritten verändert werden.

Da der Eingang des Motorboards aber eine Spannung $u_{Motor,in}$ im Wertebereich $\pm 5V$ erwartet, ist noch eine Anpassschaltung (Signal–Conditioning) nötig, weil der Motor sonst nur mit zu geringer Spannung und nur in eine Richtung drehen könnte. Das PWM–Signal wird daher verstärkt und durch eine Offset–Spannung in den geforderten Bereich verschoben (Abbildung 7). Außerdem

© Professoren W. Lindermeir, T. Rothermel, W. Zimmermann

wurde in die Anpassschaltung ein Tiefpassfilter für das PWM–Rechtecksignal integriert, um die vom PWM–Signal erzeugten EMV–Störungen zu reduzieren.

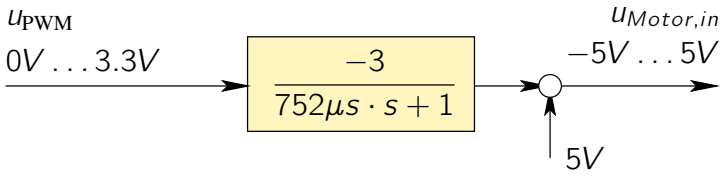


Abbildung 7: Blockschaltbild der PWM–Ansteuerung des Motors

Vorbereitungsaufgabe 4: Motorspannung bei gegebener PWM–Ansteuerung

Tragen Sie in Tabelle 2 den Zusammenhang zwischen dem PWM–Wert (8bit unsigned), dem Mittelwert der PWM–Ausgangsspannung $\overline{U_{\text{PWM}}}$ und der Motorspannung $u_{Motor,in}$ ein.

PWM–Wert	$\overline{U_{\text{PWM}}}$	$u_{Motor,in}$
0		
128		
255		

Tabelle 2: PWM–Rechenwert und Motorspannung (stationär)

3.2.3 Repeating Timer und ADC–Messung von Strom– und Tachospannung

Ein Repeating Timer des RP2040 wird verwendet, um periodisch nach der Abtastzeit $T = 1000\mu s$ einen Interrupt auszulösen. In der *repeating_timer_callback()*–Funktion erfolgt der Start der AD–Wandlungen für die Spannung des Tachogenerators und des Stromsensors (vergl. Kapitel 3.1.2). Auch diese Signale werden durch Anpassschaltungen gefiltert und an den Spannungsbereich $(0 \ldots 3.3V)$ des Analog–Digital–Umsetzers angepasst (Abbildung 8 und 9). Verwendet werden die ADC–Pins 26 (Tachogenerator) und 27 (Strom). Das Wandlungsergebnis ist rechtsbündig mit 12–bit Auflösung. Für jedes Signal wird der Median aus drei Messungen gebildet, um Messausreißer zu unterdrücken.

Nachdem die Werte des Stroms und des Tachogenerators eingelesen wurden, werden die Werte des Quadratur–Encoders eingelesen. Dabei wird die Position der Welle in Steps (400 Steps pro Umdrehung der Welle) in der Variablen *current_substep_int64* gespeichert. In der Größe *rpm_substepencoder* wird die Drehzahl der Welle in U/min ermittelt.

Anschließend werden je nach Zustand des *control_state*–Automaten in der *repeating_timer_callback()*–Funktion folgende Funktionalitäten abgearbeitet:

© Professoren W. Lindermeir, T. Rothermel, W. Zimmermann

- Der Regelalgorithmus für den Positionsregler,
- der Regelalgorithmus für den Drehzahlregler,
- der manuelle Betrieb des Motors sowie dazu jeweils passend die
- Berechnung der Sollwerte für den Sollwert–Generator.
- Es werden anzuzeigende Werte für das Soft–Scope an den Entwicklungs–PC geschickt und
- eine eventuelle Überlastung des Motors abgefangen.
- Schließlich wird der final ermittelte pwm–Wert an das PWM–Modul übergeben.

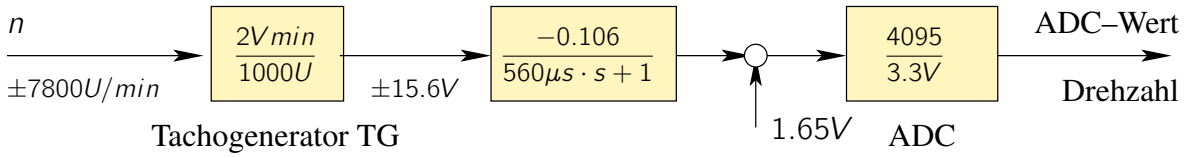


Abbildung 8: Blockschaltbild der Signal–Anpassschaltung für die Tachospannung

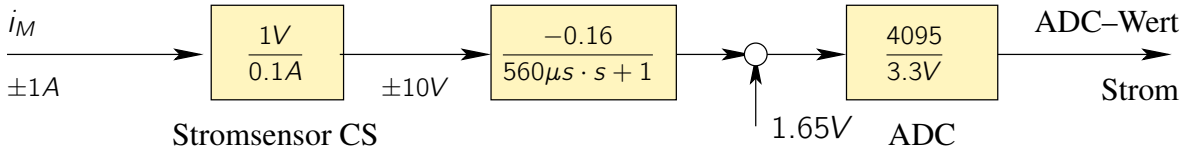


Abbildung 9: Blockschaltbild der Signal–Anpassschaltung für den Stromsensor

Vorbereitungsaufgabe 5: ADC–Werte bei gegebenen Drehzahlen und Strömen

Tragen Sie in Tabelle 3 den Zusammenhang zwischen Drehzahl bzw. Strom und den jeweiligen ADC–Messwerten ein.

© Professoren W. Lindermeir, T. Rothermel, W. Zimmermann

Drehzahl n	ADC–Wert
$-7800\frac{U}{min}$	
$-7000\frac{U}{min}$	
$0\frac{U}{min}$	
$+7000\frac{U}{min}$	206
$+7800\frac{U}{min}$	

Strom i_M	ADC–Wert
$-1A$	
$0A$	
$+1A$	

Tabelle 3: ADC–Rechenwerte bei gegebenen Drehzahlen und Strömen (stationär)

4 Modellierung und Parameteridentifikation

4.1 Simulationsmodell für die Strecke

In Abbildung 10 ist die Strecke mit dem Gleichstrommotor als Blockschaltbild dargestellt. Die Reibung ist als viskose, d.h. drehzahlproportionale Reibung mit dem Reibbeiwert d angenommen. Bei der Drehzahl n bzw. Drehkreisfrequenz ω sollten Sie immer auf die verwendete Einheit achten. Möglich sind $[\frac{U}{sec}]$ für n, $[\frac{rad}{sec}]$ für ω oder für den Menschen am anschaulichsten n in $[\frac{U}{min}]$. Dabei ist die letzte Einheit zwar für Menschen am anschaulichsten, allerdings ist dies keine SI–Einheit und sollte daher in Simulationsmodellen vermieden werden, weil sie nicht mit den SI–Einheiten anderer Größen zusammenpasst. In unserem Modell des Motors verwenden wir die Drehkreisfrequenz $\omega = 2\pi n$ mit der Einheit $[\frac{rad}{sec}]$. Allerdings wird außerhalb des eigentlichen Modells des Motors die Drehzahl in der Einheit $[\frac{U}{min}]$ angegeben. Somit können auch die Sollwerte des Drehzahlreglers in dieser Einheit verwendet werden. Daher sollen alle Drehzahlen außerhalb des eigentlichen Motormodells in $[\frac{U}{min}]$ angegeben werden.

© Professoren W. Lindermeir, T. Rothermel, W. Zimmermann

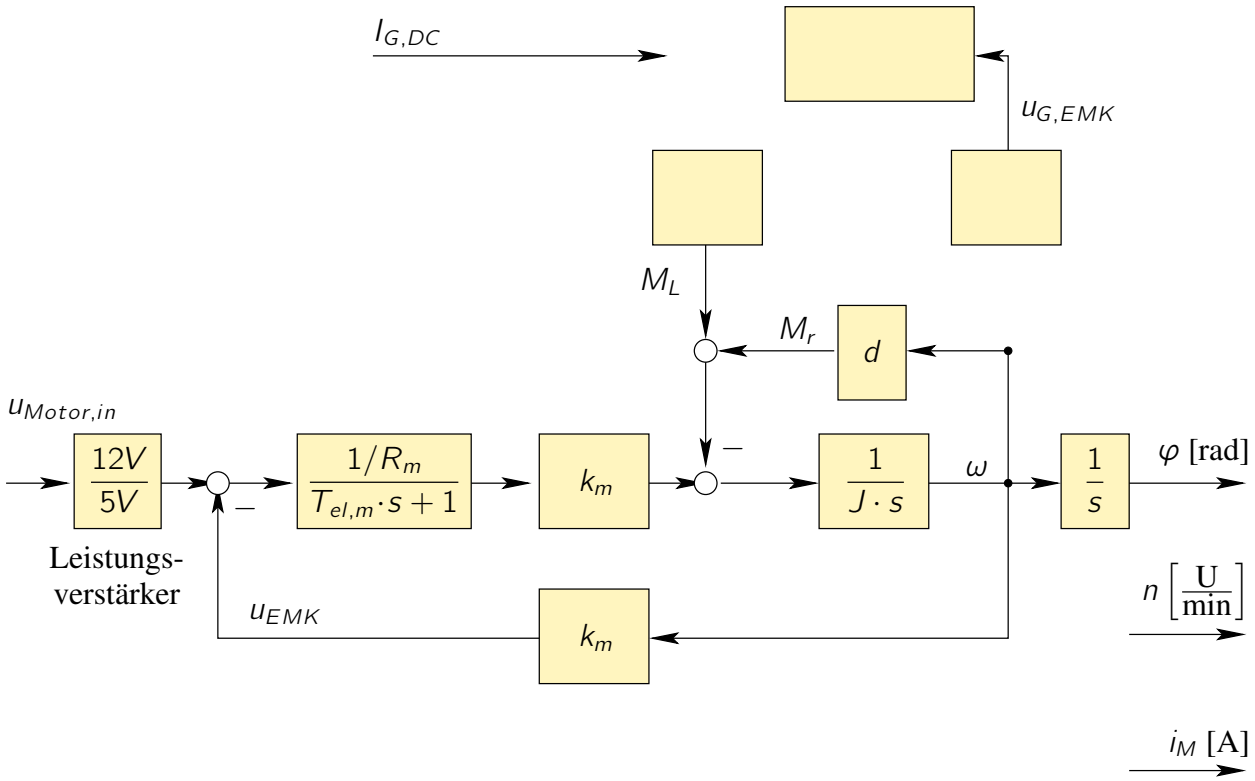


Abbildung 10: Blockschaltbild der Strecke mit den Eingängen $u_{Motor,in}$ und Generator(gleich)strom $I_{G,DC}$ als Störgröße sowie den Ausgängen Motorstrom i_M [A], Drehzahl n [U/min] und Position φ [rad].

Vorbereitungsaufgabe 6: Blockschaltbild der Strecke

Vervollständigen Sie in Abbildung 10 zunächst die Anschlüsse für n und i_M.

Beim elektrischen Teil des Motors werden die Parameter R_m und $T_{el,m}$ verwendet. Geben Sie diese Werte allgemein und mit Zahlenwert an. Nehmen Sie für die Induktivität den Wert $L = 2mH$ an. Der Widerstand R_m im Motormodell setzt sich aus dem Wicklungswiderstand R (siehe Tabelle 1) und dem Widerstand R_{shunt} des Stromsensors (siehe 3.1.2) zusammen.

$R_m =$ und $T_{el,m} =$

Die oberen drei, bisher unvollständig angeschlossenen Blöcke im Blockschaltbild sollen den Generator modellieren, der das Last–Störmoment für den Motor erzeugt. Wie schon im Ersatzschaltbild in Abbildung 1 und im Simulink–Modell (Seite 8) beschrieben, kann der Generator entweder aus einer Konstantstromquelle $I_{G,DC}$ gespeist oder an eine Glühlampe als Lastwiderstand ($R_{Lampe} = 12,4\Omega$) angeschlossen werden.

- Wenn die Konstantstromquelle verwendet wird, wird zusätzlich zum Reibmoment M_r ein Belastungsmoment $M_L = k_m \cdot I_{G,DC}$ auf den Motor wirken.

© Professoren W. Lindermeir, T. Rothermel, W. Zimmermann

- Falls dagegen die Glühlampe aktiv ist, wird das Belastungsmoment

  $$M_L = k_m \frac{\frac{1}{R_g}}{sT_{el,g}+1} \cdot \underbrace{k_m \cdot \omega}_{u_{G,EMK}:\text{induzierteSpg. am Generator}}$$

  mit $R_g = R + R_{Lampe} =$ und $T_{el,g} = \frac{L}{R_g} =$

  wirksam. Berechnen Sie die Zahlenwerte für R_g und $T_{el,g}$.

Tragen Sie in die entsprechenden Blöcke im Blockschaltbild nach Abbildung 10 die Übertragungsfunktionen und Verstärkungsfaktoren zur Beschreibung des Generators ein und zeichnen Sie die entsprechenden Verbindungsleitungen ein. Die Umschaltung zwischen den beiden Fällen für M_L soll durch einen Schalter erfolgen.

Hinweise zur Erstellung der Graphen Um sinnvolle Graphen zu erzeugen und auch die Graphen auf die interessierenden Bereiche zu zoomen, ist es sinnvoll, die Abbildungen mit einem kleinen Skript, das auf die jeweilige Aufgabe anzupassen ist, zu erzeugen. Dies basiert darauf, dass die Simulink–Modelle die jeweiligen Signale über *To Workspace*–Blöcke nach Matlab zurückschreiben. Beispielhaft könnten Sie ein Skript wie das folgende nutzen:

```
figure(1);
plot(t, n);
axis([0.98 1.05 2950 3150]);
 %Skalierung der Achsen; die vier Zahlen bedeuten:
 % x-Achse Beginn; x-Achse Ende; y-Achse Beginn; y-Achse Ende
xlabel('Beschriftung der x-Achse');
ylabel('Beschriftung der y-Achse');
title('Titelzeile');
grid;
saveas(gcf,'dateiname.pdf');
```

Vorbereitungsaufgabe 7: Simulationen der Strecke

Im Simulink–Modell *Strecke.slx* fehlen die Blöcke zur Modellierung des Generators noch, die Sie soeben in der Abbildung ergänzt haben. Bauen Sie diese fehlenden Blöcke in das Modell ein. Die für das Modell benötigten Parameter finden Sie in der Datei *strecke_params_m.m*, mit der dieses Modell vor der Simulation initialisiert werden muss. Führen Sie dann zur Verifikation Ihrer Änderungen am Modell Simulationen mit folgenden Werten durch:

- Simulationsdauer $t_e = 5sec$
- Sprung der Eingangsspannung zur Zeit $t_w = 1sec$
- Sprung der Stromquelle zur Zeit $t_v = 3sec$:

© Professoren W. Lindermeir, T. Rothermel, W. Zimmermann

Folgende Fälle sollen simuliert werden:

a. Sprung von $u_{Motor,in}$ von 0V auf 4V, kein Strom $I_{G,DC}$ durch den Generator.

b. Sprung von $u_{Motor,in}$ von 0V auf 4V, Generator an Lampe.

c. Sprung von $u_{Motor,in}$ von 0V auf 4V, Generator an Stromquelle mit Sprung von 0 auf $I_{G,DC} = +200mA$.

d. Sprung von $u_{Motor,in}$ von 0V auf 4V, Generator an Stromquelle mit Sprung von 0 auf $I_{G,DC} = -200mA$.

Vergleichen Sie die Ergebnisse Ihrer Simulationen für den Fall b mit denjenigen, die in Abbildungen 11 und 12 dargestellt sind. **Falls Ihre Simulationen andere Ergebnisse als die in diesen beiden Abbildungen dargestellten liefern, müssen Sie Ihren Fehler suchen und korrigieren bevor Sie weiterarbeiten können! Sonst sind viele nachfolgenden Simulationen auch fehlerhaft!**

Die Simulationsdiagramme sollen in getrennten Diagrammen jeweils den Motorstrom i_M und die Motordrehzahl n darstellen. Die Diagramme erstellen Sie am besten mit dem Matlab `plot()`–Befehl. Die zugehörigen Signale werden aus dem Simulink–Modell durch die To Workspace–Blöcke exportiert. Sehen Sie sich zur Erstellung des Laborberichts auch den Matlab–Befehle `saveas()` an. **Vergessen Sie die Beschriftung der beiden Achsen der Diagramme nicht und geben Sie an, zu welcher Aufgabe die jeweilige Abbildung gehört!**

Versuchen Sie die Simulationsergebnisse zu verstehen und beantworten Sie folgende Fragen:

Warum ist der Endwert des Stroms im Fall a nicht Null, obwohl es kein Störmoment gibt?

Antwort:

Warum ist die Enddrehzahl im Fall b deutlich kleiner als im Fall a?

Antwort:

Welche unterschiedliche Wirkung hat das Störmoment im Fall c und im Fall d?

Antwort:

© Professoren W. Lindermeir, T. Rothermel, W. Zimmermann

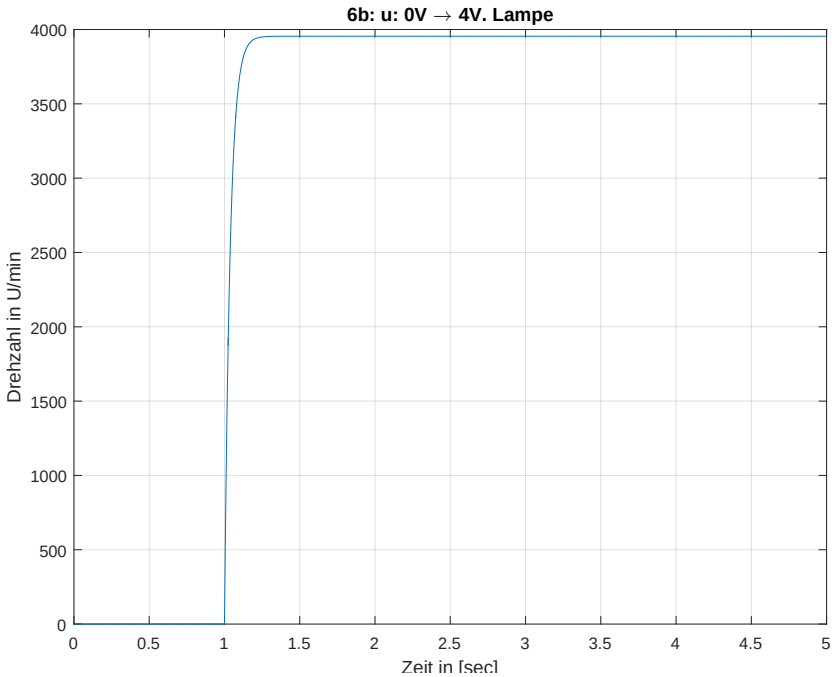


Abbildung 11: Drehzahlverlauf zu Vorbereitungsaufgabe 7.b

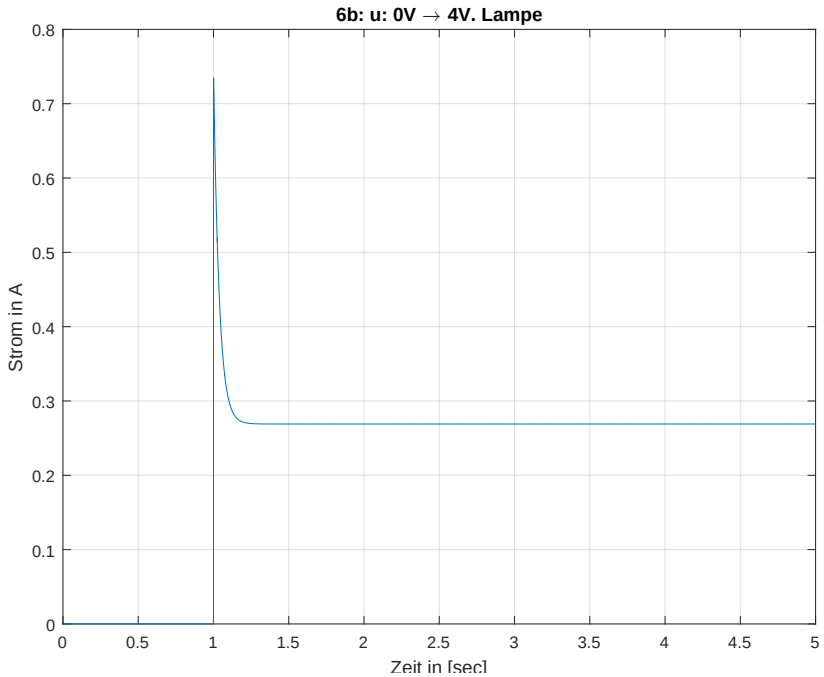


Abbildung 12: Stromverlauf zu Vorbereitungsaufgabe 7.b

© Professoren W. Lindermeir, T. Rothermel, W. Zimmermann

Vorbereitungsaufgabe 8: Nur für Linux – Vorbereiten der Toolkette

Um die Toolkette auf Ihrem Linux–Rechner zu installieren, tippen Sie die folgenden Kommandos in ein Terminal:

```
$ wget https://raw.githubusercontent.com/raspberrypi/\
pico-setup/master/pico_setup.sh
$ chmod +x pico_setup.sh
$ cp pico_setup ... wohin es installiert werden soll
$ ./pico_setup.sh
```

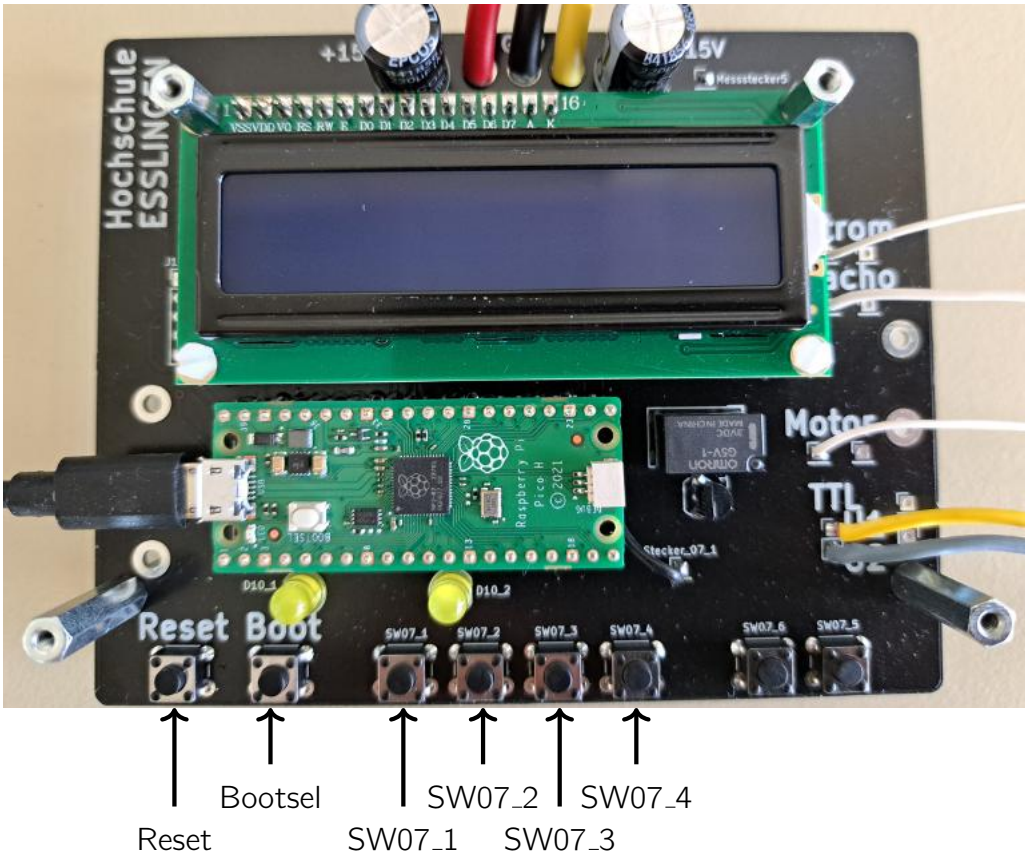


Abbildung 13: Bedienelemente des Boards; die beiden Taster rechts sind im Labor ohne Funktion

© Professoren W. Lindermeir, T. Rothermel, W. Zimmermann

Laboraufgabe 0: Überprüfen des Laboraufbaus

Unter Linux Das Projekt Compilieren und auf das Board laden:

- Öffnen Sie ein neues Terminal, damit (*.bashrc* ausgeführt wird).
- Im Projektordner z.B. *Lab1_uC* alle Dateien im *build*–Verzeichnis löschen (wenn der Ordner existiert)
- Falls der Ordner *build* nicht existiert, legen Sie in an.
- Die Datei *CMakeLists.txt_Linux* in *CMakeLists.txt* umbenennen (Achtung; das müssen Sie in jedem der weiteren Projekte auch machen!).
- dann im *build*–Verzeichnis folgende Kommandos eingeben:

```
$ cmake ..
$ make
```

- USB Kabel anschließen
- RP Pico in Bootsel mode versetzen (siehe Abbildung 13; Reset und Boot Tasten drücken; dann erst Reset und danach Boot Taste loslassen)
- Dann erscheit das Board ähnlich einem eingesteckten USB–Stick am Entwicklungs–PC.
- Mounten Sie den RP Pico und kopieren Sie die Datei *build/motorboard.uf2* in das gemountete Verzeichnis
- Danach startet die Software auf dem Pico und der Pico unmountet sich automatisch

Aufsetzen der Tookkette unter Windows Sie müssen die Toolkette auf dem Laborrechner aufsetzen. Dazu gehen Sie in den folgenden Schritten vor:

- im Projektordner z.B. *Lab1_uC* das *build*–Verzeichnis löschen (wenn der Ordner existiert)
- Die Datei *CMakeLists.txt_Windows* in *CMakeLists.txt* umbenennen (Achtung; das müssen Sie in jedem der weiteren Projekte auch machen!).
- Ordner `C:\Programme2\.pico-sdk` in das Verzeichnis `C:\Benutzer\<Benutzername>` bzw. `C:\Users\<username>` kopieren.
- Start von Visual Studio Code (VSC)
- USB Kabel anschließen

© Professoren W. Lindermeir, T. Rothermel, W. Zimmermann

- in VSC TAB *File* den Menüpunkt *Open Folder* ... aufrufen
- dort den Projektpfad auswählen (z.B. `D:\projektname\Lab1_uC`)
- in dem Popup-Fenster *Trust the authors* ... auswählen und mit *Yes,* ... bestätigen
- die Activity Bar (Default: Aktivitätsleiste auf der linken Seite von VSC) identifizieren
- das Aktivitätsleisten-Element *Extension* auswählen und eine kurze Zeit die Aktualisierung abwarten
- dort im Suchfenster *cortex–Debug* eingeben und installieren
- im Popupfenster *Trust & install* auswählen
- danach das gleiche Vorgehen mit *Raspberry Pi Pico*
- beim Auftreten des VSC Popup *No Python found* ... den Button *Install uv and Python* auswählen
- weiter im Suchfenster *CMake Tools* eingeben und installieren
- Popupfenster *It is recommended to reconfigure* ... ignorieren
- weiter im Suchfenster *C/C++ DevTools* eingeben und installieren
- das Aktivitätsleisten–Element *Explorer* auswählen
- danach das Aktivitätsleisten–Element *Raspberry Pi Pico Project* auswählen
- Downloading und Installation abwarten
- danach *Compile Project* auswählen
- RP Pico in Bootsel mode versetzen (siehe Abbildung 13; Reset und Boot Tasten drücken; dann erst Reset und danach Boot Taste loslassen)
- Beim ersten Aufruf im *Open Quick Acces* Fenster (VSC oberste Menüzeile) Projekt *motorboard* auswählen
- dann *Run Project (USB)* auswählen

Erste Bedienung der Software zur Steuerung/Regelung des Elektromotors Stellen Sie mit Hilfe der Bedientasten (sh. S.39) den Manual–Modus ein und aktivieren Sie den eingebauten Rechteckgenerator, um Spannungssprünge an den Motor anzulegen. Starten Sie in Matlab das *SoftScope* (sh. S.37), um den Drehzahlverlauf zu messen. Beim Starten des *SoftScopes* müssen Sie die richtige serielle Schnittstellen herausfinden (`SoftScope(0)` listet vorhandene Schnittstellen auf; Unter Windows z.B. Lesen von com3 bzw. unter Linux /dev/ttyACM0. Wichtig ist die zugewiesene Portnummer, die als *assigned to port number xx*

© Professoren W. Lindermeir, T. Rothermel, W. Zimmermann

ausgegeben wird. Sie starten dann das SoftScope mit `SoftScope(xx)`, wobei xx ein Integer ist).

Wenn Sie die Drehzahländerungen hören (!) und sehen können, funktioniert der Aufbau korrekt und Sie können mit den eigentlichen Laboraufgaben beginnen.

4.2 Parameteridentifikation

Unter **Systemidentifikation** versteht man das Finden einer zu den Messungen passenden Systemstruktur und deren Parametrierung. Man kann z.B. die Sprungantwort oder den Frequenzgang einer Strecke messen und dann eine Übertragungsfunktion suchen, die zu den Messwerten passt.

In der Praxis erhält man allerdings wesentlich bessere Ergebnisse, wenn man, wie im vorliegenden Fall, ein auf Gleichungen aufbauendes physikalisches Modell für die Strecke aufstellt. Dann muß man „nur“ diejenigen Parameter des physikalischen Modells, die nicht in Datenblättern zu finden sind, durch **Parameteridentifikation**, d.h. aus Messungen, gewinnen. Durch den Vergleich der Messergebnisse mit Simulationsmodellen kann und muß man in der Praxis auch überprüfen, ob das Simulationsmodell korrekt ist.

Der „Trick“ bei der Parameteridentifikation besteht darin, genau solche Messungen durchzuführen, die die Wirkung jedes Modellparameters möglichst unabhängig von den anderen Parametern zeigen. Im Folgenden werden wir diskutieren, mit welchen Messungen die Motorparameter identifiziert werden können. Einen Teil dieser Messungen werden Sie selbst im Labor durchführen.

4.2.1 Elektrischer Teil des Motors: R, L

Am einfachsten sind die elektrischen Daten des Motors zu bestimmen. Mit einem Ohmmeter können wir prüfen, ob der im Datenblatt angegebene Wert für R korrekt ist. Da es Exemplarstreuungen in der Fertigung gibt und der Wicklungswiderstand von der Temperatur abhängt, sind kleinere Abweichungen zu erwarten.

Laboraufgabe 1: Messen des Widerstandes R

Messen Sie den Ohmwiderstand R im Ankerkreis des Motors mit einem digitalen Multimeter. Schließen Sie dazu die Klemmen des DMMs direkt an den Ankerkreis des Motors an. Achten Sie darauf, dass sich die Welle des Motors dabei nicht dreht.

Gemessener Widerstandswert: $R =$

Versuchen Sie zu verstehen, warum sich die Welle dabei nicht drehen darf. Sie können mit einem Finger die Welle bewegen und dabei sehen, wie sich die Widerstandsmessung verfälscht.

Leider können wir mit den uns im Labor zur Verfügung stehenden Messgeräten die Induktivität L des Motors nicht nachmessen. Wir verwenden daher wie oben einen geschätzten Wert von $L = 2mH$.

© Professoren W. Lindermeir, T. Rothermel, W. Zimmermann

4.2.2 Reibung

Um die Reibung zu bestimmen, kann man die aufgenommene elektrische Leistung $u_{motor,in} \cdot i_M$ im Leerlauf bei verschiedenen Drehzahlen n messen. Diese „Leerlauf“leistung wird im Wesentlichen benötigt, um die Reibung zu überwinden. Aus der Leerlaufleistung lässt sich dann das Reibmoment M_R und der Reibbeiwert d berechnen, wie bereits in Vorbereitungsaufgabe 2 beschrieben.

Die Messergebnisse in Abbildung 14 zeigen allerdings, dass wir nicht nur viskose Gleitreibung, d.h. rein drehzahlproportionale Reibung haben. Die realen Messdaten (rote Kurve) zeigen einen Versatz zwischen den positiven Drehzahlen und den negativen Drehzahlen. Bei sehr kleinen Drehzahlen um 0 dominiert die Haftreibung (Coulombsche Reibung). Durch eine Ausgleichsrechnung kann eine approximierende Gerade (blaue Kurve) ermittelt werden, die die viskose Gleichreibung annähert. Für die Drehzahlregelung bei höheren Drehzahlen können wir von reiner Gleitreibung ausgehen (blaue Kurve). Bei der Positionsregelung sind aber durch die stark nichtlineare Haftreibung Probleme zu erwarten.

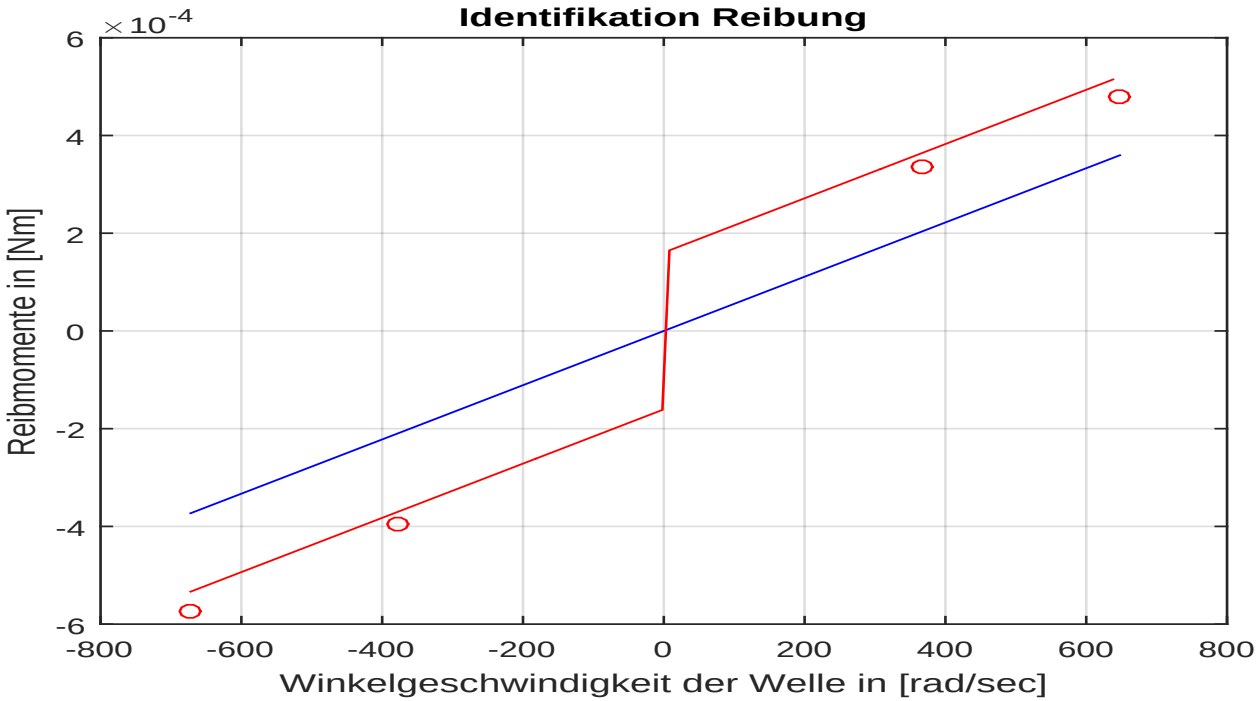


Abbildung 14: Ergebnisse der Reibungsmessung (rot: Messdaten, blau: rein viskose Reibung)

4.2.3 Trägheitsmoment

Zur Identifikation des Trägheitsmoments J gehen wir wie folgt vor: Wir messen die Sprungantwort des Motorstroms i_M bei Sprüngen der Eingangsspannung u_{PWM}, indem wir den PWM–Wert zwischen 220 und 180 umschalten. Beim Schalten des PWM–Werts von 180 auf 220 steigt der Strom

© Professoren W. Lindermeir, T. Rothermel, W. Zimmermann

zunächst stark an und fällt dann wieder ab, weil sich die Drehzahl des Motors (betragsmäßig) erhöht und damit die induzierte Spannung u_{EMK} vergrößert und der Eingangsspannung entgegenwirkt. Der betrachtete Teil des Blockschaltbildes ist in Abbildung 15 dargestellt.

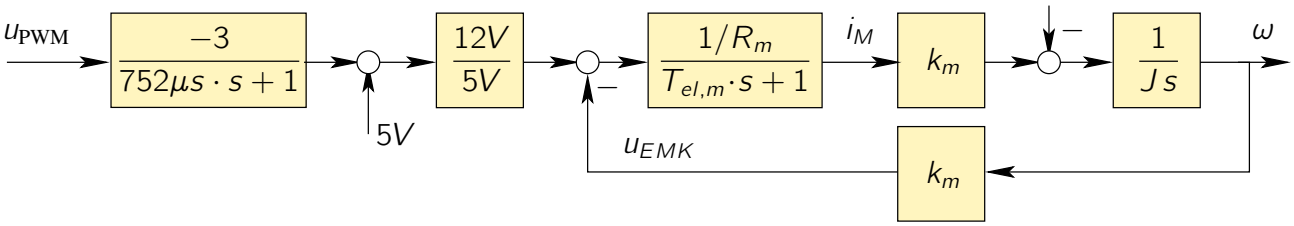


Abbildung 15: Blockschaltbild zur Identifikation des Trägheitsmoments

Das Trägheitsmoment J bestimmt dabei, wie schnell sich die Drehzahl erhöht und damit der Strom abfällt. Abbildung 16 zeigt den zeitlichen Verlauf des Motorstroms i_M. Der gemessene Stromverlauf ist in rot dargestellt. Überlagert ist in blau das Ergebnis einer entsprechenden Simulation mit dem Simulink–Modell *Regelkreis_ZK.slx*. Dabei wurde J in der Simulation so lange verändert, bis der simulierte Verlauf von i_M mit dem gemessenen Verlauf bestmöglich übereingestimmt hat. Für das kleinere Trägheitsmoment haben wir so $J_{K(lein)} = 0,7 \cdot 10^{-6} Nmsec^2$, für das größere Trägheitsmoment $J_{G(ross)} = 1.5 \cdot 10^{-6} Nmsec^2$ gefunden.

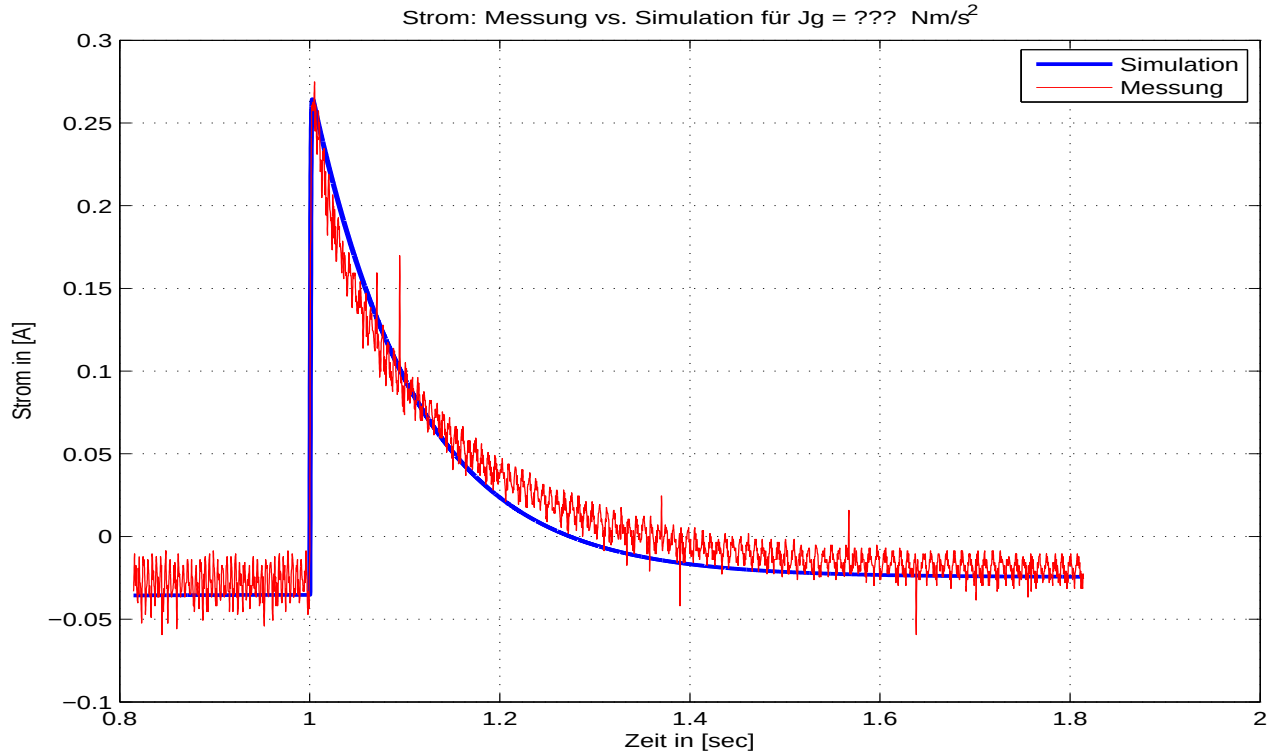


Abbildung 16: Ergebnisse der Messung des Trägheitsmoments (rot: Messdaten, blau: Simulation)

© Professoren W. Lindermeir, T. Rothermel, W. Zimmermann

4.2.4 Motorkonstante k_M

Als letzte Größe des Gleichstrommotors bestimmen wir die Motorkonstante k_m. Dazu verwenden wir folgendes „Experiment“:

- Wir klemmen den Motor vom Leistungsverstärker ab, d.h. $i_M = 0$ (vergl. Abb. 1) und bringen ihn von außen auf eine konstante Drehzahl, indem wir ihn über den Generator antreiben.
- Mit einem Digitalvoltmeter messen wir dann die Spannung an den Klemmen des Motors. Da kein Strom fließt, ist die Klemmenspannung gleich u_{EMK}.
- Gleichzeitig ist nach dem Blockschaltbild in Abb. 10 $u_{EMK} = k_m \cdot \omega = k_m \cdot 2\pi n$.
  $\Rightarrow k_m = \left| \frac{u_{EMK}}{2\pi n} \right|$ (Achtung: u_{EMK} in V, n in $\frac{U}{sec}$ nicht $\frac{U}{min}$!).

Laboraufgabe 2: Bestimmung der Motorkonstante k_m

Führen Sie die oben beschriebene Messung in folgenden Schritten durch:

- Entfernen Sie die Steckbrücke SB zwischen Generator und Lampe (vergl. Abb. 3).
- Entfernen Sie die Verbindung vom Leistungsverstärker zum Motor und schließen Sie stattdessen den Leistungsverstärker an den Generator an.
- Messen Sie mit einem Digitalvoltmeter die Klemmenspannung U_{EMK} des Motors.
- Laden Sie das Mikrocontroller–Projekt auf das Board und führen Sie es aus. Wählen Sie über die Bedientaster den Handbetrieb (manual), stellen Sie über die Taster die PWM–Werte nach Tabelle 4 ein und tragen Sie die nachfolgend ermittelten Messdaten spaltenweise ein.
  - Die vom DMM gemessene Spannung U_{EMK}.
  - Im Zustand *disp_only* wird in der zweiten Zeile des LCD–Displays sowohl die über die Encoder–Signale des Positionsgebers mit Hilfe des Encoders ermittelte Drehzahl n_E als auch die vom Mikrocontroller über den Tachogenerator analog gemessene Drehzahl n_T angezeigt. Tragen Sie diese Werte in die Zeilen n_T und n_E in der Tabelle ein.
  - Messen Sie mit dem Digitalspeicheroszilloskop DSO ein Encoder–Signal des Positionsgebers und ermitteln Sie die Periodendauer T_O dieses Signals (O steht für Oszilloskop; vergl. Abb. 4). Da der Geber 100 Pulse je Umdrehung liefert, ist die Motordrehzahl $n_O = \frac{60}{100 \cdot T_O}$, Ergebnis in $\frac{U}{min}$ mit T_O in sec. Die Formel liefert kein Vorzeichen für die Drehzahl. Wählen Sie das Vorzeichen von n_O so, dass es dasselbe ist wie das Vorzeichen von U_{EMK} und tragen Sie diesen Wert auch in die Tabelle ein.

© Professoren W. Lindermeir, T. Rothermel, W. Zimmermann

Falls die von Ihnen berechneten Werte für n_O stark von den beiden anderen Werten für abweicht, überprüfen Sie Ihre Berechnung. Möglicherweise haben Sie einen Fehler mit den Einheiten gemacht.

– Berechnen Sie aus den Messergebnissen von n_O (!) und U_{EMK} den Mittelwert von k_m (Einheit nicht vergessen!). Vergleichen Sie Ihr Ergebnis mit dem Wert aus der Simulationsdatei *params_m.m*. Bei einer Abweichung von mehr als ca. ±20% haben Sie sich möglicherweise verrechnet oder bei den Einheiten vertan.
Siehe oben: $k_m = \left|\frac{u_{EMK}}{2\pi n}\right|$ (Achtung: u_{EMK} in V, n in $\frac{U}{sec}$ nicht $\frac{U}{min}$!).

PWM–Wert:	220	180	75	35	Einheit
U_{EMK}					V
n_T					U/min
n_E					U/min
T_O					μsec
n_O					U/min
k_m					?

Ergebnis: Mittelwert von $k_m =$

Tabelle 4: Messung Motorkonstante k_m und Überprüfung des Tachogenerators

Im weiteren Verlauf des Labors soll der Motor untersucht werden. Daher müssen Sie den **Leistungsverstärker wieder an den Motor anschließen**.

4.3 Validierung des Simulationsmodells, Drehzahlmessgenauigkeit

Nachdem nun alle Parameter des Modells mit Hilfe von Messungen identifiziert wurden, ist die Modellbildung abgeschlossen. Um sicherzustellen, dass Modell und reales System übereinstimmen, sollten aber noch einige weitere Messungen und Simulationen miteinander verglichen werden. So zeigt Abb. 17 den Vergleich des Drehzahlverlaufs n_T des Tachogenerators zur simulierten Drehzahl bei einem Sprung der PWM–Ansteuerung 180 → 220. Während der zeitliche Verlauf recht gut übereinstimmt, gibt es offensichtlich einen Drehzahlfehler, der zudem von der Höhe der Drehzahl abhängt.

© Professoren W. Lindermeir, T. Rothermel, W. Zimmermann

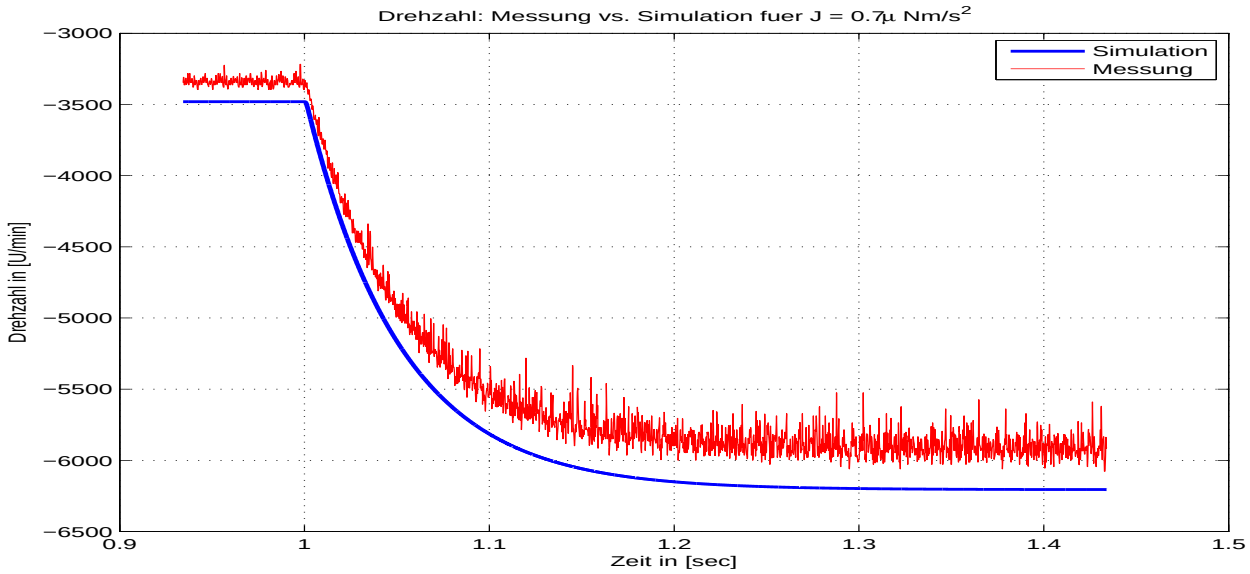


Abbildung 17: Validierung des Modells gegen Messung: Gemessener Drehzahlverlauf n_T (rot) bzw. simulierter Drehzahlverlauf (blau) bei PWM–Ansteuerung 180 $\rightarrow$ 220.

Einer der Hauptunterschiede zwischen Simulation und Mikroprozessor ist die Drehzahlerfassung. Bei der Messung des „analogen“ Drehzahlsignals n_T über die Anpassschaltung und den ADC–Eingang des Mikroprozessors entstehen Offset- und Verstärkungsfehler. Das „digitale“ Drehzahlsignal n_E dagegen, bei dem die Impulsperiode der Encoder–Signale des Positionsgebers über den Encoder im Mikroprozessor gemessen wird, ist dagegen ziemlich genau.

Laboraufgabe 3: Überprüfung der analogen Drehzahlerfassung

Bei der vorigen Laboraufgabe haben Sie in Tabelle 4 sowohl die aus den digitalen Encoder–Signalen des Positionssensors ermittelte Drehzahl n_E als auch die vom Mikroprozessor über den analogen Tachogenerator gemessene Drehzahl n_T erfasst.

- Zeichnen Sie ein Diagramm $n_T(n_E)$ und versuchen Sie, eine Ausgleichsgerade zu bestimmen. Theoretisch müsste die Kennlinie exakt durch den Ursprung gehen und eine Steigung von 1 haben. Bestimmen Sie mit der Ausgleichsgerade die Messfehler:

  Offset bei Drehzahl 0 in U/min: $n_{ofs} =$

  Steigung k_n:

  Sie können für dieses Diagramm das Matlab–Skript `drehzahlMessung.m` verwenden. Editieren Sie das Skript und tragen Sie Ihre Messwerte ein, bevor Sie das Skript ausführen. Das Skript berechnet den Offset und die Steigung der Ausgleichsgerade mit Hilfe der Matlab–Funktion `polyfit()` und zeichnet die Ausgleichsgerade mit Hilfe der Matlab–Funktion `polyval()`.

© Professoren W. Lindermeir, T. Rothermel, W. Zimmermann

- Nehmen Sie dieses Diagramm in Ihren Laborbericht auf.

Regelungen können nicht genauer sein als die verwendeten Sensoren, selbst wenn die Regelung theoretisch keine bleibende Regelabweichung hat. Wenn die Genauigkeit des Sensors nicht ausreicht, könnte man z.B. wie hier die Sensorkennlinie ausmessen und den Sensorfehler im Reglerprogramm durch eine Korrektur berücksichtigen. Alternativ kann man einen anderen Sensor verwenden. Im vorliegenden Fall ist es naheliegend, die Drehzahl vom Encoder zu verwenden.

5 Auslegung des Drehzahlreglers in der Simulation

5.1 Dimensionierung des PI–Drehzahlreglers

Der Drehzahlregler ist mit Hilfe des Matlab–Skripts *auslegung_DrehzahlRegler_PI_m.m* auszulegen. Der offene Regelkreis setzt sich zusammen aus:

- Der Übertragungsfunktion *k_cond_pwm* nach Abb. 7 für die PWM–Ansteuerung vom Eingang PWM bis zum Ausgang $U_{Motor,in}$
- Der Streckenübertragungsfunktion *G_Strecke_n* nach Abb. 10 vom Eingang $U_{Motor,in}$ bis zum Ausgang n (ohne den Generator für das Belastungsmoment).
- Dem Tachogenerator mit der Übertragungsfunktion *k_cond_n* nach Abb. 8 vom Eingang n über den ADC und die Rückrechnung der Drehzahl im Mikrocontroller.

Als Regler verwenden wir einen PI–Regler, den wir mit Polkompensation und Vorgabe der Phasenreserve (Nyquist–Verfahren) auslegen. Bei der Polkompensation wird die langsamste Zeitkonstante der Regelstrecke durch die Reglernullstelle kompensiert.

Vorbereitungsaufgabe 9: Dimensionierung des Drehzahlreglers

Sehen Sie sich das M–File *auslegung_DrehzahlRegler_PI_m.m* an und versuchen Sie den Inhalt zu verstehen.

Dimensionieren Sie die Verstärkung k_{Rn} des Drehzahlreglers (für $J = J_k$) mit Hilfe des Matlab–Skripts so, dass der **offene** Regelkreis eine Phasenreserve von $\varphi_R = 60^0$ hat. Die Nachstellzeit T_n wird im Matlab–Skript automatisch so eingestellt, dass die langsamste Streckenzeitkonstante kompensiert wird. Das Bodediagramm des offenen Regelkreises wird durch die Matlab–Funktion *margin()* gezeichnet und die Phasenreserve als *Pm* in das Diagramm eingetragen. Verändern Sie die Verstärkung im Matlab–Skript solange, bis Sie die geforderte Phasenreserve erhalten.

Welche Werte ergeben sich für die Proportionalverstärkung, die Nachstellzeit und die Durchtrittskreisfrequenz des Drehzahlreglers?

$k_{Rn} =$ $T_n =$ $\omega_{D,n} =$

Tragen Sie diese Werte k_{Rn} und T_n auch in die Parameter–Initialisierung *params_m.m*

© Professoren W. Lindermeir, T. Rothermel, W. Zimmermann

ein. Speichern Sie dieses Skript und führen Sie es aus, damit die Werte auch bei der nachfolgenden Simulation des Modells in Simulink zur Verfügung stehen.

Begrenzung der Stellgröße des Drehzahlreglers:
Wie aus der Vorlesung bekannt, ist die Stellgröße eines realen Reglers immer beschränkt. Die Ansteuerung des PWM–Ausgangs des Mikrocontrollers kann nur mit Werten von $0 \ldots 255$ erfolgen. Daher ist ein entsprechendes Sättigungselement im Simulink–Modell vorgesehen. Der Ausgang des Reglers besitzt damit einen Wertebereich von $-127,5 \ldots +127,5$, weil nachfolgend der Arbeitspunkt durch einen Offset von $127,5$ eingestellt wird (siehe Simulink–Modell *Regelkreis_ZK.slx*).

Weil der Drehzahlregler ein PI–Regler ist, werden wir in der Praxis bei großen Sollwertsprüngen ein schlechtes Einschwingverhalten bekommen, sobald der Regelkreis dynamisch in die Begrenzung geht. Wir müssen für den Drehzahlregler daher eine entsprechende **Anti–Windup–Maßnahme** mit den entsprechenden Grenzen einbauen.

5.2 Verifikation des Drehzahlreglers in der Simulation

Der entworfene Drehzahlregler soll nun in der Simulation verifiziert werden. Die Parameter des Simulink–Modells *Regelkreis_ZK.slx* sind im M–File *params_m.m* angegeben. Bevor Sie simulieren, sollten Sie dieses Skript ausführen. Stellen Sie die Schalter und den Step–Block für den Sollwert des Drehzahlreglers im Simulink–Modell so ein, dass der Drehzahlregler mit den gewünschten Sollwertsprüngen entsteht. Sie sollten jeweils bis ca. $t_e = 2.5sec$ simulieren.

Vorbereitungsaufgabe 10: Führungssprungantwort — Kleinsignal
Simulieren Sie einen Sollwertsprung von 3000 auf 3100 U/min ohne Störung zur Zeit $t_w = 1sec$. Die Ergebnisse sollten ungefähr so aussehen wie der Führungssprung aus der vorherigen Aufgabe, als Sie den Drehzahlregler ausgelegt haben. Fügen Sie die Zeitverläufe von Drehzahl und Strom in Ihren Bericht ein. **Achten Sie auf eine sinnvolle Skalierung der Achsen, damit man den interessierenden Teil der Zeitverläufe auch sehen kann. Die interessierende Sprungantwort findet im Zeitintervall um 1sec statt!!**

Vorbereitungsaufgabe 11: Führungssprungantwort — Großsignal
Simulieren Sie einen Sollwertsprung von 1000 auf 6000 U/min zur Zeit $t_w = 1\,\text{sec}$ ohne Störung. Fügen Sie die beiden Zeitverläufe von Drehzahl und Strom in Ihren Bericht ein. **Achten Sie auch hier auf sinnvolle Skalierung der Achsen!**

6 Implementierung des PI–Reglers auf dem Mikrocontroller

Um den Drehzahlregler auf das reale Mikrocontrollerboard zu portieren, sind drei Schritte notwendig.

© Professoren W. Lindermeir, T. Rothermel, W. Zimmermann

6.1 Umwandlung in einen zeitdiskreten PI–Algorithmus

Wie bereits in Regelungstechnik 1 besprochen, kann ein zeitkontinuierlicher PI–Regler

$$G_R(s) = \frac{U(s)}{E(s)} = k_R \cdot (1 + \frac{1}{sT_n})$$

durch folgenden rekursiven Algorithmus im Zeitbereich nachgebildet werden, der periodisch im Abtastraster berechnet werden muss:

$$u_I(k) = u_{\text{I,alt}} + k_R \cdot \frac{T}{T_n} \cdot e(k)$$

$$u(k) = k_R \cdot e(k) + u_I(k)$$

Dabei ist T die Abtastzeit und es wird angenommen, dass die Abtastzeit deutlich kleiner ist als die Nachstellzeit T_n, d.h. es wird ein quasi–zeitkontinuierlicher Entwurf vorausgesetzt. Die erste Zeile berechnet den I–Anteil aus dem I–Anteil des letzten Zeitschritts und dem aktuellen Inkrement, die zweite Zeile addiert dazu den P–Anteil. In einer C–Funktion muss die Variable $u_{\text{I,alt}}$ persistent implementiert werden, z.B. als statische lokale Variable, und auf z.B. den Wert 0 initialisiert werden.

Die Anti–Windup–Begrenzung sowie die Stellgrößenbeschränkung mit den Grenzwerten u_{min} und u_{max} ist wie folgt möglich:

```
if(u > u_max) {
    u = u_max;          /* u_I_alt bleibt */
}else if(u < u_min) {
    u = u_min;          /* u_I_alt bleibt */
}else{
    u_I_alt = u_I;      /* Index Shift    */
}
```

6.2 Test des zeitdiskreten PI–Algorithmus in C im Simulink–Modell

Vorbereitungsaufgabe 12: Implementieren des PI–Reglers in C und Test in Simulink

Das Simulink–Modell *Regelkreis_ZD_C.slx* enthält einen C/C++–Block (Simulink S–Function), in dem die C–Funktion *zPIawu_n.c* verwendet wird. Ergänzen Sie diese Funktion so, dass sie den geforderten Anti–Windup–PI–Regler implementiert. Die Abtastzeit sowie die Reglerparameter werden der *S–Function* in der Maske des Reglerblocks im Simulink–Modell übergeben.

Vergessen Sie nicht, die C–Funktion mit *mex zPIawu_n.c* in eine ausführbare Datei zu übersetzen, bevor Sie das Simulink–Modell simulieren. Zusätzlich müssen Sie auch die C–Funktion für den Positionsregler mit *mex zP_pos.c* entsprechend übersetzen. Simulieren Sie Ihren Regler bei der voreingestellten Abtastzeit. Sie sollten bei den verschiedenen Sprungantworten (3000 auf 3100 U/min und 1000 auf 6000 U/min) ähnliche – aber nicht unbedingt identische – Ergebnisse bekommen, wie im letzten Abschnitt.

© Professoren W. Lindermeir, T. Rothermel, W. Zimmermann

Zu dieser Vorbereitungsaufgabe müssen Sie nichts abgeben.

6.3 Portieren des zeitdiskreten PI–Algorithmus auf das Mikrocontroller–Board

Damit sind alle Vorarbeiten abgeschlossen und der Reglerentwurf kann auf dem Mikrocontroller implementiert werden.

Laboraufgabe 4: Portieren und Testen des Reglers auf dem Mikrocontroller–Board

Öffnen Sie das Mikrocontroller–Projekt im Verzeichnis *Lab1_uC*. Die Source–Dateien, die dieses Projekt verwendet, liegen immer im Verzeichnis *src*, die Header–Dateien im Verzeichnis *include*.

Implementieren Sie den Drehzahlregler. Sie müssen dazu im Mikrocontroller–Projekt in der Datei *main.c* in der Funktion *rpmControl()* den entsprechenden C–Code ergänzen und die Reglerparameter eintragen. Diese Funktion wird durch die Funktion *repeating_timer_callback* im Abtastraster zyklisch aufgerufen. Verwenden Sie die C–Datei, die Sie in der letzten Vorbereitungsaufgabe entwickelt haben, als Vorlage:

- Die Variablen, die Sie beim Implementieren der Regler verwenden sollen, sind im Quelltext im Kommentar beschrieben. **Lesen Sie diesen Kommentar!**
- Der PI–Regler soll wieder mit einer **Anti–Windup**–Maßnahme ausgestattet sein.
- Die Abtastzeit des Regler ist auf 1000μsec (*SAMPLINGTIME_US*) eingestellt.

Wenn Ihr Programm fehlerfrei compiliert wurde, laden Sie das Mikrocontroller–Projekt auf den Mikrocontroller. **Vergewissern Sie sich, dass Sie den Motor und nicht den Generator vom Mikrocontroller ansteuern!** Wenn der Regler funktioniert, sollten Sie dies hören. Wenn die **Dioden der Vierquadrant–Anzeige wild blinken**, schalten Sie das Board bitte möglichst **schnell aus** und debuggen Sie Ihre Implementierung.

Messen Sie den Drehzahlverlauf mit Hilfe des Softwareoszilloskops (sh. S.37). Fügen Sie die Sprungantworten in Ihren Laborbericht ein.

7 Positionsregler

7.1 Dimensionierung und Test des Positionsreglers mit Matlab/Simulink

Als Positionsregler verwenden wir einen P–Regler, den wir mit Vorgabe der Phasenreserve (Nyquist–Verfahren) auslegen.

Der Regler wird mit Hilfe des M–Files *auslegung_Positionsregler_P_m.m* dimensioniert. Im Matlab–Skript wird zunächst die Streckenübertragungsfunktion der Positionsregelstrecke bestimmt.

© Professoren W. Lindermeir, T. Rothermel, W. Zimmermann

Darüber hinaus wird die Übertragungsfunktion der Rückführung (Positionsdekoder als P–Glied modelliert) angegeben. Die übrige Vorgehensweise im Matlab–Skript entspricht derjenigen bei der Auslegung des Drehzahlreglers oben.

Vorbereitungsaufgabe 13: Dimensionierung und Simulation des Positionsreglers

Vorbereitungsaufgabe 13.1 Dimensionierung
Damit der Positionsregler praktisch kein Überschwingen zeigt, dimensionieren Sie den Positionsregler mit Hilfe des Matlab–Skripts so, dass sich eine Phasenreserve von $\varphi_R = 85^0$ einstellt. Sehen Sie sich das M–File an und versuchen Sie seinen Inhalt zu verstehen, bevor Sie das Skript verwenden. **Stellen Sie sicher, dass Sie in die Datei** *params_m.m* **die Werte** k_{Rn} **und** T_n **eingetragen haben, die sich bei der Dimensionierung des Drehzahlreglers ergeben haben, bevor Sie den Positionsregler auslegen!**
Welche Werte ergeben sich für den Proportionalbeiwert und die Durchtrittskreisfrequenz des Positionsreglers?

$k_{Rpos} =$ $\omega_{D,pos} =$

Tragen Sie diesen Wert von k_{Rpos} in das Matlab–Skript *auslegung_Positionsregler_P_m.m* ein und verifizieren Sie Ihre Dimensionierung.

Wenn das System das gewünschte Führungsverhalten zeigt, tragen Sie den Wert für k_{Rpos} auch in die Parameter–Initialisierungsdatei *params_m.m* ein.

Vorbereitungsaufgabe 13.2 Simulation
Der Positionsregler soll nun in der Simulink–Simulation verifiziert werden. Verwenden Sie dazu das Simulink–Modell *Regelkreis_ZK.slx*.

Begrenzung der Stellgröße des Positionsreglers:
Um zu vermeiden, dass der unterlagerte Drehzahlregler mit einem realitätsfernen Sollwert angesteuert wird, wird der Ausgang des überlagerten Positionsreglers (= Sollwert des Drehzahlreglers) im Simulink–Modell auf sinnvolle Drehzahlwerte begrenzt. Die Begrenzung erfolgt so, dass sich ein maximaler Drehzahlsollwert von $\pm 7800 U/min$ ergibt.

Stellen Sie die Schalter und den Step–Block für den Sollwert des Positionsreglers im Simulink–Modell so ein, dass der Positionsregler einen Sollwertsprung von 0 auf $10U$ ($U = Umdrehungen$) zur Zeit $t_w = 1sec$ ohne Störgröße ausführt.

Fügen Sie das Simulationsergebnis (Position, Drehzahl und Strom) mit sinnvoll skalierten Achsen Ihrem Laborbericht hinzu.

7.2 Implementierung der Positionsregelung auf dem Mikrocontroller–Board

Laboraufgabe 5: Überprüfen der Drehzahlregelung und Einbau der Positionsregelung

Bauen Sie den **P–Positionsregler** in Ihr Programm ein. Die entsprechende Stelle in der Funktion *positionControl()* in der Datei *main.c* sowie die Variablen, die Sie lesen bzw. schreiben

© Professoren W. Lindermeir, T. Rothermel, W. Zimmermann

dürfen, sind entsprechend kommentiert.

Laden Sie das Programm auf das Mikrocontroller–Board und testen Sie nochmals die **Drehzahlregelung** mit den Drehzahlsprüngen wie oben. Die Drehzahlverläufe dürfen sich gegenüber der vorigen Messung nicht wesentlich verändert haben.

Schalten Sie nun über die Bedientasten auf die **Positionsregelung** um und stellen Sie den Automatik–Betrieb ein, bei dem Sollpositionssprünge zwischen 0 und 10 Umdrehungen gemacht werden. Der schnelle Wechsel zwischen den beiden Positionen sollte an der Welle des Motors sichtbar (und hörbar!) sein. **Falls die Dioden der Vierquadrantenanzeige wild blinken, schalten Sie das Board bitte möglichst schnell wieder aus** und überprüfen Sie Ihre Implementierung.

Messen Sie den Positions- und Drehzahlverlauf mit Hilfe des Softwareoszilloskops (sh. S.37). Fügen Sie das Bild des Softwareoszilloskops in Ihren Laborbericht ein.

Damit ist der erste Laborversuch beendet. Die ausführliche Analyse der Positionsregelung folgt im zweiten Laborversuch.

© Professoren W. Lindermeir, T. Rothermel, W. Zimmermann

8 Anhang: Analyse von Messdaten

8.1 Software-Oszilloskop

Interne Werte des Mikrocontrollerprogramms, z.B. die mit dem ADC gemessene Motordrehzahl, die Solldrehzahl oder die Istposition, können über die serielle Schnittstelle im Echtzeitbetrieb an den PC übertragen und in Matlab angezeigt werden. Dazu wird **im Mikrocontrollerprogramm** am Ende der Funktion `repeating_timer_callback()`, die im Abtastraster alle $T = 1000\mu sec$ ausgeführt wird, die Funktion

```
SoftScope((int16) groesse_1, (int16) groesse_2);
```

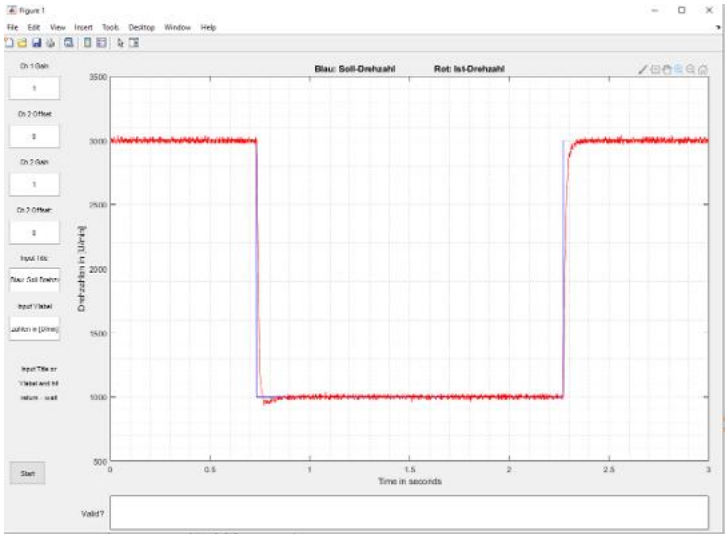
aufgerufen. Beachten Sie, dass die übertragenen Werte in 16bit Integerzahlen konvertiert werden. Für eine Übertragung von Gleitkommazahlen oder von mehr als zwei Werten innerhalb der Abtastzeit ist die serielle Schnittstelle (115kbit/sec) zu langsam.

Das Mikrocontrollerboard ist über eine USB–Schnittstelle mit dem PC verbunden werden. Über diese Schnittstelle wird das Board geflashed. Im Betrieb kann über diese Schnittstelle auch die Standardausgabe des Mikrocontrollers an den PC übertragen werden.

In der Matlab-Konsole muss die folgende Funktion für das Software-Oszilloskop ausgeführt werden:

```
SoftScope(0)    % listet vorhande Schnittstellen auf
SoftScope(25)   % erwartet Daten von serieller Schnittstelle 25
```

Die dargestellten Zeitsignale sind die vom Mikrocontroller-Programm gesendeten 16bit-Integerwerte. Das Bildschirmfenster zeigt einen Zeitbereich von einigen Sekunden an und wird laufend aufgefrischt. Um Details näher analysieren zu können, stoppen Sie das Oszilloskop über den Start/Stop-Button im Programmfenster und zoomen Sie im Bild mit den üblichen Matlab-Werkzeugen.

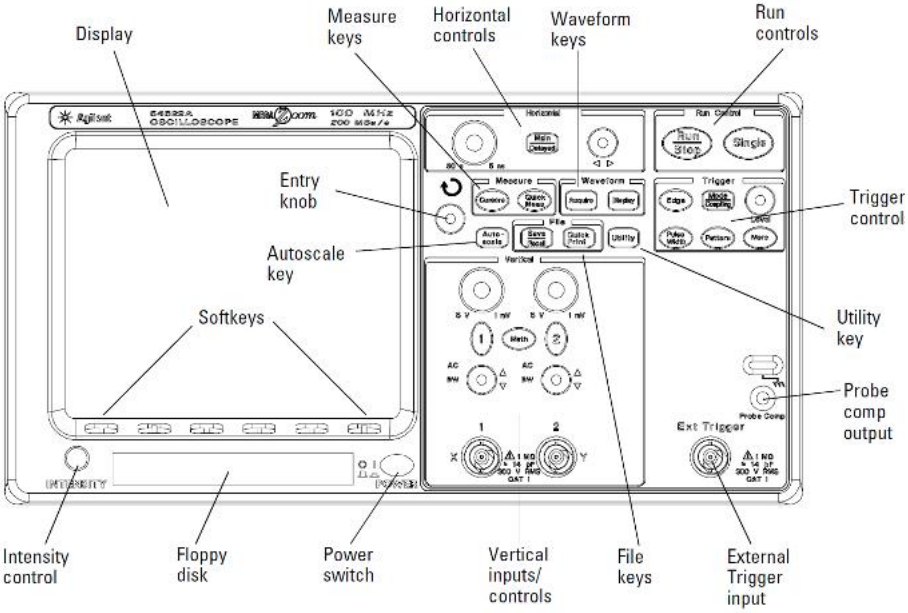


Achtung: Falls das Software-Oszilloskop hängen bleibt, schließen Sie das Fenster und starten Sie es neu. Eventuell müssen Sie anschließend `clear all` ausführen oder sogar Matlab neu starten.

© Professoren W. Lindermeir, T. Rothermel, W. Zimmermann

8.2 Bedienung des Digital–Speicher–Oszilloskops

Die Bedienung erfolgt über Tasten (**Hardkeys**) und Drehknöpfe (**Knobs**) auf der Vorderseite des Gerätes sowie Menütasten (**Softkeys**) unterhalb des Displays. Wenn eine Menütaste mehrere Optionen hat, wählt man diese über den Drehknopf **Entry knob**.



Grundeinstellungen:

- **Tastköpfe anschließen** und mit Messignalen verbinden. Zugehörige Kanäle über Hardkeys 1 und 2 einschalten.
- Hardkey **Autoscale** drücken: Stellt Kanalverstärkung und Trigger automatisch ein, funktioniert aber nur bei „schnellen“ Signalen, z.B. den TTL-Signalen des Positionsgebers. Für Messung von Sollwertsprüngen der Drehzahl mit dem Tachogebersignal nicht geeignet.
- Erneutes Drücken von Autoscale schaltet auf die vorige manuelle Einstellung zurück.

Kanalverstärkung und Zeitablenkung:

- Die **Verstärkung** kann für jeden Kanal an den großen Drehknöpfen im Bedienfeld Vertical eingestellt werden. Mit den darunterliegenden kleinen Drehknöpfen kann für jeden Kanal der **Offset**, d.h. die vertikale Lage, verstellt werden.
- Die **Zeitablenkung** wird mit dem großen Drehknopf im Bedienfeld Horizontal eingestellt.
- Die eingestellten Werte (V/Div bzw. s/Div) werden im Display in der oberen Zeile angezeigt.

Trigger:

- Hardkey **Edge** wählt man das Menü im Display aus. Mit Softkey **Rising** $\Uparrow$ **bzw. Falling** $\Downarrow$ kann die steigende oder fallende Signalflanke zum Triggern ausgewählt werden. Mit den Softkeys 1 bzw. 2 wählt man den **Triggerkanal** aus. Der **Triggerpegel** kann über den Drehknopf Level im Bedienfeld Trigger eingestellt werden.
- Hardkey **Mode** wählt das Menü im Display aus. Über den Softkey Mode wählt man Auto oder Normal (für langsame Signale wie Sollwertsprünge besser geeignet). Bei **Coupling** sollte DC ausgewählt werden.

© Professoren W. Lindermeir, T. Rothermel, W. Zimmermann

9 Anhang: Bedienoberfläche des Mikrocontroller–Programms

Bedientasten:

- Taste SW07_1:
  Umschalten zwischen Drehzahlregelung (R), Positionsregelung (P) und Handbetrieb (M)
- Taste SW07_2:
  Umschalten zwischen automatischer Sollwerterzeugung (eingebauter Rechteckgenerator) oder Sollwertvorgabe über SW07_3/SW07_4.
- Taste SW07_3:
  z.B. Sollwert vergrößern
- Taste SW07_4:
  z.B. Sollwert verringern

Anzeige im LCD–Display obere Zeile:

- Im RPM–(Drehzahlregler)–Modus:
  R: w=Solldrehzahl e=Regelfehler (beide in U/min)
- Im Positionsregler–Modus:
  P: w=Sollposition e=Regelfehler (beide in Ticks)
- Im Manual–(Hand)–Modus:
  M: u=manuell vorgegebene Stellgröße nT: Tacho–Generator–Drehzahl (in U/min)

Anzeige im LCD–Display untere Zeile:

- Bedienhinweise zu den Tasten SW07_3, SW07_4 zur Sollwertvorgabe
- Nach *AutoStim* wird die Rechenzeit der Reglerroutine angezeigt
- Bedeutung der Drehzahldarstellungen
  - nT : Tachogenerator: Drehzahl vom analogen Messwert
  - nE : Encoder: Drehzahl vom Quadraturpositionsgeber

© Professoren W. Lindermeir, T. Rothermel, W. Zimmermann